



Stock look-ahead prediction and portfolio management using fuzzy transformer architecture and reinforcement learning

Hui Jin^a, Qi Cao^{b,*}, Chai Quek^a

^a College of Computing and Data Science, Nanyang Technological University, 639798, Singapore

^b Department of Statistics and Data Science, Faculty of Science, National University of Singapore, 117546, Singapore

HIGHLIGHTS

- A fuzzy Transformer architecture is proposed as a type of fuzzy deep neural network.
- It is employed for the multi-day look-ahead price predictions in financial markets.
- It enhances the system interpretability based on attention weights and fuzzy rules.
- Overall framework using fuzzy Transformer architecture is for portfolio management.

ARTICLE INFO

Keywords:

Fuzzy transformer architecture
Look-ahead predictions
Predicted technical indicator
Portfolio management

ABSTRACT

In the modern Artificial Intelligence (AI) centric world, the unmatched data processing prowess of deep learning is often overshadowed by its black box nature, making its predictions seem arcane in certain industrial sectors. It is beneficial to understand the rationale behind the decisions or predictions by the networks. The need for interpretability gradually becomes desirable. This research introduces an innovative type of fuzzy neural network (FNN), incorporating the favourable performance offered by a Transformer network with a five-layer fuzzy logic system. The proposed fuzzy Transformer architecture combines the attention layers in the encoder-decoder structure with human understandable fuzzy rules, which maintains favorable predictive performance while enhancing its interpretability. Based on the multi-day look-ahead prediction results of fuzzy Transformer architecture, a predicted Moving Average Convergence Divergence (MACD) technical indicator is derived, which greatly alleviates the latency problem inherent in the vanilla MACD indicator. A framework is developed using the fuzzy Transformer architecture to take in time series financial data after preprocessing, normalization and feature selections. With the predicted MACD indicator from the fuzzy Transformer architecture, a reinforcement learning (RL) agent is revised and trained for portfolio management comprising three financial assets. Experiments are conducted on fuzzy Transformer architecture to forecast the multi-day look-ahead prices, up to 10 days look-ahead. Prediction performance on individual assets is evaluated and compared with those of another four deep learning neural networks. The proposed fuzzy Transformer architecture achieves higher average improvement results by 3.2–154.5%. The predicted MACD performance results are also compared with four deep learning models and the vanilla MACD. The proposed fuzzy Transformer outperforms most benchmark models on multiple metrics. The performance of portfolio management with the RL agent is also analyzed, with the tunable hyper parameters. Observed from the experiment results, the developed system consisting of revised feature selection algorithm, fuzzy Transformer architecture and revised RL agent is able to improve the performance in trading tasks and portfolio management.

1. Introduction

Nowadays, artificial intelligence (AI) gains broader adoptions. Large

datasets, powerful computational hardware and various deep learning architectures have led to groundbreaking achievements to solve problems in many fields. As a type of recurrent neural network (RNN), long

* Corresponding author.

E-mail addresses: hjin003@e.ntu.edu.sg (H. Jin), qi.cao8@nus.edu.sg (Q. Cao), ashcquek@ntu.edu.sg (C. Quek).

<https://doi.org/10.1016/j.asoc.2026.115862>

Received 10 May 2024; Received in revised form 23 June 2026; Accepted 29 June 2026

Available online 30 June 2026

1568-4946/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

short-term memory (LSTM) is able to address the vanishing gradient problem, when the input sequence is too long [1]. LSTM can erase, write and read the long-term information stored in the cells. These are controlled by forgetting gates, input gates and output gates. The state of these gates is computed based on the current context at each time step. Bidirectional LSTM (Bi-LSTM) is an extended architecture of LSTM with two layers: one as forward direction to process data sequence from past to future, the other as backward direction to process data from future to past [2].

Transformers gradually become popular due to the favorable performance [3]. Transformer-based architecture has further improved model performance in natural language processing (NLP) [4,5], computer vision (CV) [6,7], time series data classification [8], and other domains [9]. Stock price forecasting is considered as the time series data analytics task. The stock market has seen a surge in adopting deep learning models for prediction and trading decisions. These models have proven valuable in areas like time series data analytics, price movement forecasting, algorithmic trading, and portfolio management [10]. As a result, traders or investors often merge the predictions from these models with their own insights to make better trading decisions. Transformer models have also been reported in the financial market predictions. A Transformer encoder Gated Recurrent Unit (GRU) is reported for stock price forecasts with technical indicators and news sentiments analysis [9]. A Transformer encoder-based attention network is introduced for stock movement predictions [11]. A Transformer model is presented to predict the prices of stock market indexes [12]. The price predictions of market indexes are conducted with a series decomposition Transformer and period-correlation method [13].

In deep learning, defining useful features can be challenging. A feature that benefits one model might become noise for another. There are often many features found in large datasets involved in deep learning neural networks. Feature selection is usually employed to choose appropriate features and reduce the number of input variables for data preprocessing. There are various feature selection methods reported in the literature. Correlation coefficient method selects features with high correlation to the target variable [14]. Variance threshold method calculates the variance of each feature. It then removes all features with a lower variance than the threshold [15].

Feature selection can be based on feedback from some machine learning algorithms, such as forward selection and backward elimination methods [16]. The forward selection method starts without features and adds the feature at each step that provides the most significant improvement in model performance. The process ends when the addition of new features does not improve performance further. While the backward elimination method starts with all features and removes the feature per step that has the least impact or even deteriorates model performance. It stops when the removal of features starts harming the performance. Monte Carlo evaluative selection (MCES) is another method between the forward selection and backward elimination, that takes advantage of both. MCES deletes one feature at a time and then judges the importance of the deleted features, based on the positive or negative feedback results from the model for selected features in each iteration [17]. There are also other feature selection methods, such as using L1 regularization to train the model while performing feature selection [18], or conducting feature compressions on large data to become lower dimensional space [19], etc.

As deep learning models are often seen as "black boxes", it is challenging to interpret how and why they produce an output or decision [10,20,21]. It may result in hesitations for people to fully trust AI [22]. It may increase skepticism even more when inaccurate results are derived. It could become a major obstacle to applying AI algorithms in certain mission critical use cases, where the transparent reasoning procedures are expected for audit or evidence purposes. Fuzzy systems embed expert knowledge with membership functions. It attempts to uncover underlying fuzzy logic and rules using human languages. This method allows for a better understanding of the relationship between input and

output domains while dealing effectively with uncertainty. However, fuzzy systems do not always perform the same levels of performance as those of deep learning [22]. It heavily relies on prior knowledge, hindering end-to-end learning. Some research has reported to integrate the strengths of both deep learning neural networks and fuzzy systems to be fuzzy neural networks (FNN) [23,24]. There are a lot of risks and uncertainties in a stock market environment. Some FNN models are reported for stock market applications in the literature [10,21]. There are various types of deep neural networks to be incorporated with fuzzy systems in the FNN for financial market prediction tasks.

Currently, the method of analyzing attention matrices of Transformers through heatmap visualization is popular in the fields of NLP and CV. Transformers are capable of capturing correlations between past and future observations in sequences among time series data through self-attention mechanisms, although long-term dependences in long-sequence time series could lead to large computational complexity and high computational costs with black-box nature [25]. Transformer architecture is considered suitable in time series forecasting tasks with promising accuracy. Inspired by this, a system is proposed to analyze fuzzy rules more finely. This research first adopts a revised MCES algorithm for feature selection, where four types of neural networks including Multi-Layer Perceptron (MLP), CNN, LSTM and Transformer, are employed to derive the average feedback and select most relevant features based on different datasets. The derived features are fed into a propose fuzzy Transformer model combining the Transformer-based deep learning architecture and fuzzy logic, as a powerful learner to achieve an interpretable end-to-end model leveraging performance of Transformer. Then, this proposed fuzzy Transformer architecture is employed for predicting multi-day look-ahead stock prices. The predicted look-ahead prices are utilized to derive a non-lagging Moving Average Convergence Divergence (MACD) technical indicator which is engaged in stock trading and portfolio management. We will use the predicted MACD indicator to be one of the input vector elements, to trigger an Advanced Actor Critic (A2C) reinforcement learning (RL) agent performing portfolio management.

In the financial market, deciding which stock to buy, or how much to buy at a given moment is challenging to determine. It is often not easy to train a generalized prediction model with high precision. Traders, at every moment, have to make decisions based on past information, their personal experiences, and technical indicator analysis for future trend predictions. There are various technical indicators in the financial market used for trading analysis, including Relative Strength Index (RSI) [26], Price Percentage Oscillator (PPO) [10], On-Balance Volume (OBV) [27], and MACD, etc. Each technical indicator has its own strength to detect certain price patterns, and traders may usually use one or multiple technical indicators. But MACD is one of the most popular technical indicators, due to its sensitivity to small changes of prices, and effectiveness in identifying long-term volatility trend direction and price momentum [28].

RL enables an agent to make decisions in a given environment to maximize rewards. RL is reported to be adopted in the tasks of portfolio management, to train an agent as an outstanding trader in stock trading [29], such as value-based RL algorithm and policy-based algorithm [10]. The actor-critic algorithm is a mixture of value-based and policy-based algorithms [30]. It updates two neural networks. One network is the actor network, updating the probabilities from the policy. The other network is the critic network, approximating the state-action function. Actor-critic algorithms can incorporate benefits of the value-based and policy-based algorithms. The A2C algorithm is a variance of actor-critic algorithm, which incorporates an advantage function indicating the chosen action with how much better than the average in a state [31].

Portfolio management is a challenging task to achieve favorable results, even with RL methods. In this work, we adopt and revise an A2C RL agent, including the state vector, Action function and Reward function, that incorporates a few tunable hyper parameters to aim at better rewards and portfolio performance. The revisions on the RL action and

reward could reduce the difficulty of tasks by adjusting the Softmax policy. A way to provide the RL agent with higher-quality information is also developed by integrating the look-ahead predictions from the fuzzy Transformer architecture.

Our main contributions in this paper are listed as follows.

- A system is proposed consisting of a pipeline of data pre-processing, normalization, and feature selections using the revised MCES algorithm, where the suitable input features are identified and fed into the proposed fuzzy Transformer architecture. The multi-day look-ahead prices are predicted by the fuzzy Transformer architecture, in order to derive a non-lagging MACD indicator, that is next fed to a revised A2C RL agent to perform financial portfolio management.
- A fuzzy Transformer architecture is proposed combining a five-layer fuzzy system and a Transformer with encoder-decoder structure. Fuzzified clusters are taken as the inputs to Transformer. Transformer performs learning, while fuzzy logic carries out tasks of generating the rules and assigning a weight to each rule. Transformer attention weights for the antecedents of fuzzy logic are used in the generated fuzzy rule. It enhances the interpretability of the system based on the attention weights of Transformer and the resulting fuzzy rules. It provides an alternative way to combine fuzzy logic with Transformer in the tasks of multi-day look-ahead predictions of stock prices.
- The results from the multi-day look-ahead predictions are utilized to improve a lagging technical indicator MACD, which is used by the revised A2C RL agent for portfolio management tasks, with tunable hyper parameters to potentially explore the higher returns of portfolio management.

The organization of the remaining parts of this paper is as follows.

[Section 2](#) introduces the latest related work in literature. [Section 3](#) presents the proposed fuzzy Transformer architecture for the multi-day look-ahead predictions and the revised RL for the portfolio management. Experiments are conducted to evaluate the performance of the system with a portfolio consisting of three financial assets in [Section 4](#). Conclusion and discussions are depicted in [Section 5](#).

2. Related work

In this section, related work published in recent years is discussed on applications of fuzzy Transformer architecture and attention based Transformer neural networks, especially in forecasting or prediction tasks.

A fuzzy attention network-based Transformers (FANTF) was reported in [32], to incorporate the degrees of memberships of fuzzy logic into the calculations on attention scores of Transformers. The degrees of memberships were derived from tunable fuzzy logic parameters including the center of memberships and scale of the sigmoid function. Experiment results were discussed on various time series datasets, such as weather, traffic flow, and exchange rates. It showed the improvements of prediction tasks on Mean Squared Error (MSE) and Mean Absolute Error (MAE), compared to benchmark models including Transformer and Informer models. In another work, after the raw data pre-processing, a step of fuzzy logic transformation was incorporated with a set of rules to process continuous variables into fuzzy sets, to enhance feature representation of noisy data [33]. Such fuzzy logic transformation included normalization, mean-centering, and nonlinear transformation, that was next fed into a Transformer block for prediction tasks. The hybrid model was evaluated with a dataset consisting of hourly interval samples of electricity energy consumption, that outperformed benchmark models, such as Transformers, LSTM, and CNN-GRU with lower Mean Absolute Percentage Error (MAPE) and MAE values. Time series weather forecasting tasks were performed using a Transformer with a fuzzy attention mechanism, where the fuzzy expected solutions were derived from hidden layers and context vectors at

the encoders [34]. The experiments were conducted using the weather dataset with 10-min interval data samples, and the WTH dataset with 1-hour interval data of 12 climate features, achieved low MSE and MAE forecasting performance.

A method for wildfire probability forecasting was introduced to incorporate Transformer with self-attention mechanisms and interpretable rule-driven response actions using fuzzy logic [35]. Inputs of identified features relevant to wildfire and environment were fed into the Transformer architecture, to derive the forecasted burn probability map in the next 1–5 days. Such outputs were then fed into a Mamdani-type fuzzy rule-based system to generate response actions as good interpretability in fuzzy linguistic terms, such as no action required, send alert, or evacuation advisory, etc. A multivariate fuzzy trend matching (MFTM)-Informer model was reported to incorporate two parallel forecasting paths [36]. One path was for forecasting long-term trends by MFTM from insights gained from historical experience. The other path was for forecasting short-term fluctuations component by Informer from high frequency historical fluctuation series. Good interpretability was achieved by fuzzy logic in the MFTM path, and good accuracy was obtained by the Informer path. The prediction results of both parallel paths were aggregated finally as the h -step forecasts. Experiments were conducted on three datasets: Brent crude oil prices, European Union Allowance (EUA), and Standard & Poor's 500 index (SPX), shown promising results of MAE, RMSE, MAPE, R^2 , etc. A model named FDACNet was presented combining a fuzzy dominated feature extraction phase with another self-attention and temporal convolution phase [37]. The extracted fuzzy relationship feature matrix was then fed to the self-attention and temporal convolution phase, where final features were derived from the weighted features from both phases. Experiments and accuracy scores were discussed using various datasets of traffic flow, weather, audio recognition, and Electroencephalography (EEG) signals, etc.

A fuzzy-CNN model was reported for one-day ahead forecasting of stock prices [38]. It utilized MACD histogram values to detect price turning points and split local trends first, which was followed by fuzzifications of identified three characteristics of local trends. The outputs of fuzzy logic layer were fed to CNN for the next-day forecasting. Experiments with 40 stocks in BIST (Borsa Istanbul) 100 index were performed with 79.04% accuracy obtained, better than those of ARIMA, SVM and CNN models. A combination of Automated Machine Learning (AutoML) and fuzzy logic was presented for stock price forecasting [39], where AutoML was utilized to optimize the fuzzification steps in fuzzy logic, such as selections of membership functions, optimal number of elements in fuzzy partitions and types of fuzzy sets. Its performance of RMSE and MAE was evaluated using several stock market indexes and seven stocks in S&P 500 in the timeframe of 1st July 2019–1st July 2024. The pooling layers of a CNN model were revised by incorporating an indicator-time-based spherical triangular fuzzy pooling (ITSFP) layer, that assigned different weights to pooling regions to capture various market conditions from uncertainty of financial data [40]. ITSFP enabled better interpretability using fuzzy logic compared to the black box of Max Pooling and Average Pooling methods. The accuracy of ITSFP outperformed the CNN configurations with Max Pooling and Average Pooling in predicting prices of five market indexes, such as S&P 500, DJI, NASDAQ, etc.

An encoder-decoder Transformer architecture with dual attention networks was reported in [41], one as the price attention network for forecasting future prices, the other as the nonprice attention network for supporting financial features. The model was tested using multiple financial index and stocks dataset including S&P500, NYSE, APPLE, Google, etc., obtained low results on MAE, MSE, and RMSE. A modified Transformer architecture was introduced by removing the tokenizer and embedding layer, as well as adding the sinusoidal positional encoding and layer normalization compared to traditional Transformers [42]. It reduced the computation complexity in analysing time series data, where time series energy production dataset and weather dataset were

utilized in the experiments. Lower results of MAE, MSE, and RMSE were obtained compared with other models such as LSTM, Transformer, etc. PMformer as a correlation-enhanced Informer was reported with local attention computation and causal convolution operations in the encoder, while with multi-layer perceptual cross-self-attention in the decoder [43]. Performances of MSE and MAE were improved compared to Transformers and informers, etc., that were observed from experiments conducted with six datasets including energy, transportation, public health, meteorology, and the price data of a US stock, Netflix. MEAformer as a Transformer-based model was introduced by replacing self-attention with temporal external attention mechanisms, and replacing intricate decoder layers with MLP layers [44]. It facilitated exploring potential relationships between sequences of data at lower linear complexity and higher inference speed. MEAformer was evaluated with datasets for energy, transportation, economy, weather, and disease, with lower MSE and MAE compared to benchmark models including LSTM, Informer, etc.

A CNN-Transformer-MLP model was reported for forecasting of solar and wind power output trained with 15-min intervals dataset, where short-term correlations between adjacent time steps captured by CNN were fed to the Transformer layer for capturing long-term dependencies of data, and result to be produced by MLP model [45]. Experiment results showed CNN-Transformer-MLP obtained better RMSE, MAE, MSE, and R^2 than benchmark models of CNN, Bi-LSTM, and Transformers. For traffic flow forecasting tasks, a multi-channel spatial-temporal Transformer model (MC-STTM) utilized a spatial block with multiple-layers graph convolutional networks (GCN) to extract spatial features, followed by a Transformer-based temporal block to capture long-term temporal dependencies of time series traffic data [46]. Experiments were conducted using PeMS road traffic datasets with 5-minute intervals that showed improved performances on MAE, MAPE and RMSE compared to benchmark models such as GRU, GraphWaveNet, etc. An MTL-GARCH model was reported that consisted of pipeline layers with GARCH (Generalized Auto-Regressive Conditional Heteroskedasticity)-type layer, LSTM layer, and multi-Transformer architecture layer [47]. Outputs from the GARCH layer were fed to the LSTM layer, while outputs of the LSTM layer were further fed to multi-Transformer layer. Average multi-head mechanism was utilized to calculate the mean values of multi head attention units, each of which was involved certain number of different random samples generated from 90% randomly selected dataset. A TFMADR model was reported for time series forecasting tasks by integrating temporal fusion Transformers (TFT) and an RNN-based DeepAR model [48]. TFT with self-attention mechanisms derived results of point prediction from long-term dependencies and non-linear features in time series, where the point prediction results were passed to RNN-based DeepAR for conducting probabilistic forecasting. Experiments were conducted with four datasets including Brent crude oil price, carbon emission trading, cryptocurrency price, and electricity price, where TFMADR outperformed benchmark models such as Transformers, informers, LSTM, MLP, etc. in day-ahead prediction, using several performance metrics of MAE, MAPE, RMSE, R^2 , etc.

3. Methodology

In this section, we first introduce background knowledge relevant to clustering techniques, feature selection and technical analysis indicators of financial markets. Next, we present the proposed fuzzy Transformer architecture that combines advantages of high performance of Transformer architecture and good interpretability of fuzzy logic.

3.1. Clustering techniques

Clustering is used to group data points together. It is one of the unsupervised machine learning methods. In clustering, the goal is to find natural groupings or patterns in data, where points within the same

group are more similar to each other, than to those in other groups. There are various clustering techniques in machine learning. The K-means clustering technique aims to partition the data into K clusters, where each data point belongs to the cluster with the nearest mean value [49]. K-means minimizes the sum of squared distances between data points and their cluster centroids. The advantages of K-means include easy implementation and quick computations.

Hierarchical clustering technique builds a tree-like structure of clusters by successively merging or splitting clusters, which could be bottom-up or top-down [50]. Hierarchical clustering does not require specifying the number of clusters in advance. It provides a visual representation of cluster relationships.

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is an unsupervised density-based clustering algorithm [51]. It groups data points that are closely packed together, defining clusters as dense regions separated by sparser areas. There are two parameters: ϵ denoting radius of the neighborhood; and MinPts denoting the threshold value for minimum number of points to form a cluster. Neighborhood data p means all data points within distance ϵ from p . DBSCAN can discover clusters of arbitrary shapes and identifies noise points as outliers. Gaussian Mixture Models (GMM) represents clusters as a mixture of Gaussian distributions [52]. It is a probabilistic model that assigns each data point to one or more clusters with associated probabilities. GMM is useful when the data is assumed to be generated from multiple Gaussian distributions. The advantages of DBSCAN and GMM include robustness to noise, easy implementation, and decent performance.

Deep learning based clustering techniques offer flexibility and robustness in discovering clusters when dealing with high-dimensional or complex data [53]. These methods use deep learning to learn representations of data and perform clustering based on these representations [54]. Multistage deep clustering and iterative deep clustering [55] are two types of techniques in deep learning-based clustering. Deep learning based methods usually have advantages of robustness to noise and decent performance.

3.2. Technical indicators and portfolio management

As a trend-following momentum indicator, MACD is derived from subtraction of the fast and slow exponential moving averages (EMA) where usually the short period is referred to as 12 days, and the long period as 26 days. MACD helps determine the bullish or bearish trends. Considered as a lagging indicator, the vanilla MACD indicator is calculated by EMA using the current day price and past historical prices information. The MACD histogram is another lagging indicator that tracks the rate of change of the MACD indicator [10]. The MACD histogram is derived from the MACD indicator and the 9-day EMA as a signal line. The computations of the lagging indicators of EMA, MACD and MACD histogram are shown in Eq. (1).

$$EMA_t = p_t \times k + EMA_{t-1} \times (1 - k)$$

$$MACD = EMA_{short} - EMA_{long} = EMA_{12-day} - EMA_{26-day}$$

$$MACD\ Histogram = MACD - Signal\ line = MACD - EMA_{9-day} \quad (1)$$

where EMA_t is the exponential moving average at time step t ; p_t is the price at time t ; and k denotes the weight.

The buy or sell signals can be generated using MACD histogram. When the MACD histogram becomes positive, meaning the MACD line goes from below to above the signal line, it is considered a buy signal. When the MACD histogram becomes negative, meaning the MACD line goes from above to below the signal line, it is considered a sell signal. But one drawback of trend reversal detection based on the MACD histogram generated by the lagging MACD indicator (i.e., vanilla MACD) is that the past historical price information is used. It results in delayed timing in buy or sell signals. The lagging indicator issue can be resolved if only the

future price data is used to calculate MACD, that is the perfect MACD derived from ground-truth future prices. As such, there will be some differences between the vanilla MACD and perfect MACD [10], where vanilla MACD has some latencies in signals. The difference between the vanilla MACD and perfect MACD with an example stock price data is shown in Fig. 1, highlighting the evident delay in the vanilla MACD indicator.

However, in real life, this is not feasible for using the perfect MACD as we cannot perfectly predict future look-ahead stock prices. In this work, we attempt to use the predicted look-ahead n days prices which could be with higher prediction accuracy when computing predicted EMA, MACD and the MACD histogram, as shown in Eq. (2).

$$\text{Predicted_EMA}_t = \hat{p}(t+n) \times k + \text{Predicted_EMA}_{t+n-1} \times (1-k)$$

$$\text{Predicted_MACD} = \text{Predicted_EMA}_{12\text{-day}} - \text{Predicted_EMA}_{26\text{-day}}$$

$$\text{Predicted_MACD Histogram} = \text{Predicted_MACD} - \text{Predicted_EMA}_{9\text{-day}} \quad (2)$$

where $\hat{p}(t+n)$ denotes the predicted look-ahead n days prices; t is the time step; and k denotes the weight of the predicted prices. With this way, our calculations of the predicted MACD and MACD Histogram values include both historical and future look-ahead prices data. The look-ahead n days prices $\hat{p}(t+n)$ will be forecasted using the proposed fuzzy Transformer architecture. The advantage is that it reduces the delay of the lagging MACD indicator and provides more accurate trend reversal signals.

Portfolio management in the financial market refers to the process of selecting a collection of financial assets like stocks, bonds, exchange-traded fund (ETF), and then rebalancing the weight of each asset based on changing markets. Diversification is a commonly used technique in portfolio management that spreads investments across various assets or asset classes [56]. A diversified portfolio can protect the portfolio from unfavorable movements in any single asset. Exposure to a variety of assets can tap into different performance assets with the potential for better returns. But portfolio diversification is a complex process, as managing a diverse range of assets might be overwhelming.

3.3. Our overall methodology

In this work, a fuzzy Transformer architecture is proposed by the hybrid of fuzzy logic and Transformer architecture for stock prediction and portfolio management. The overall framework is shown in Fig. 2. The time series stock price data is normalized first. The feature selection is performed next. The various feature selection methods presented in Section 1 either change the form of the original inputs, or have high computational complexity. Through some comparative analysis, MCES [17] is chosen as the feature selection method in this proposed

framework. To achieve more general features, four types of neural networks are employed: MLP, CNN, LSTM and Transformer, each of which computes the outputs of each model instance with Mask 1 and Mask 2, respectively, shown in right hand side part of Fig. 2. The evaluation results of four neural networks are combined to determine the weight of each feature. We process each model's evaluation results with Softmax function to normalize the value before summing them, as we treat the feedback results of each model equally.

After that, the proposed fuzzy Transformer architecture is employed to predict look-ahead n -days stock prices, that consists of operations of fuzzification, inferencing by the Transformer neural network and fuzzy logic, and defuzzification to derive the outputs. Next, the predicted look-ahead n -days prices are utilized to enhance the lagging MACD indicator. The look-ahead predicted prices are combined with the historical price information in the calculation to derive the predicted MACD. An A2C RL agent is trained to detect trend reversal signals with the predicted MACD and perform portfolio rebalancing dynamically.

The task of fuzzy Transformer architecture is to predict the look-ahead stock prices for the next n days. Predicting future stock prices is inherently challenging. It is a complex functional relationship between the inputs and output of the model. It is hard to get the model to fit the actual distribution of prices. The range of delta change is much smaller than the price range. Given the same model's performance (e.g., predictions within $\pm 10\%$ of the correct price), smaller errors can be obtained. Thus, the delta change (Dc) is used as a type of input features to the fuzzy Transformer architecture, i.e., the daily price variation shown in Eq. (3).

$$Dc(t) = \text{price}(t) - \text{price}(t-1) \quad (3)$$

For example, $Dc(1)$ denotes the delta change in the 1-day look-ahead, $Dc(2)$ for the delta change in the 2-days look-ahead, etc. Besides the input feature of the price delta change, another type of input features is the momentum (Mt) of stock price, which is calculated in Eq. (4).

$$Mt(t) = d(t) - d(t-1) \quad (4)$$

For example, $Mt(1)$ denotes the rate of the delta change in the 1-day look-ahead, $Mt(2)$ for the rate of the delta change in the 2-days look-ahead, etc.

The output of the fuzzy Transformer architecture is the predicted delta change after n days. With the predicted price delta changes, it is easy to derive the future look-ahead prices from the last input price value. For example, the predicted price on day $t+n$ is derived from the stock price on day t , and the predicted delta changes in the next n^{th} look-ahead day.

3.4. Proposed fuzzy transformer architecture

There are five layers in the proposed fuzzy Transformer architecture

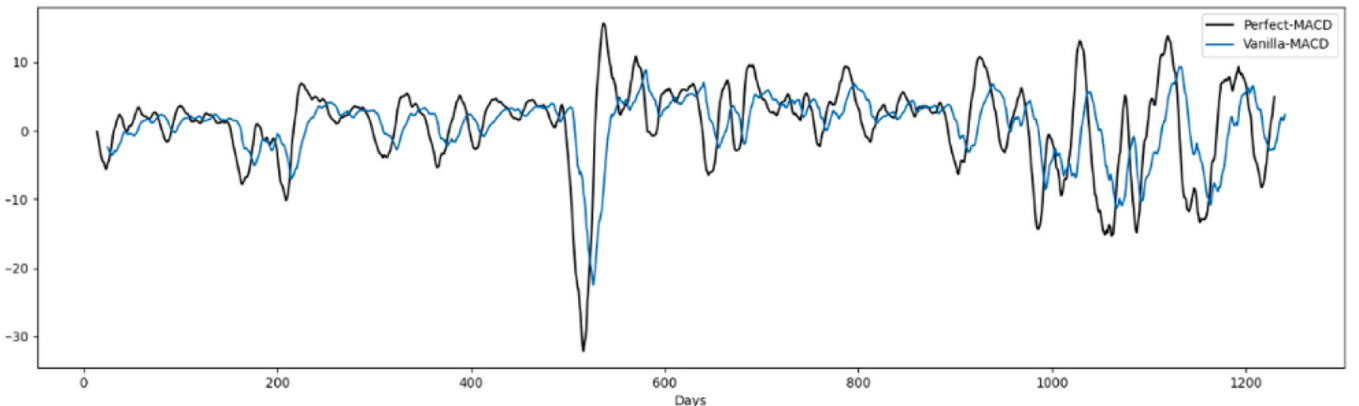


Fig. 1. Differences of the lagging vanilla MACD and non-lagging perfect MACD.

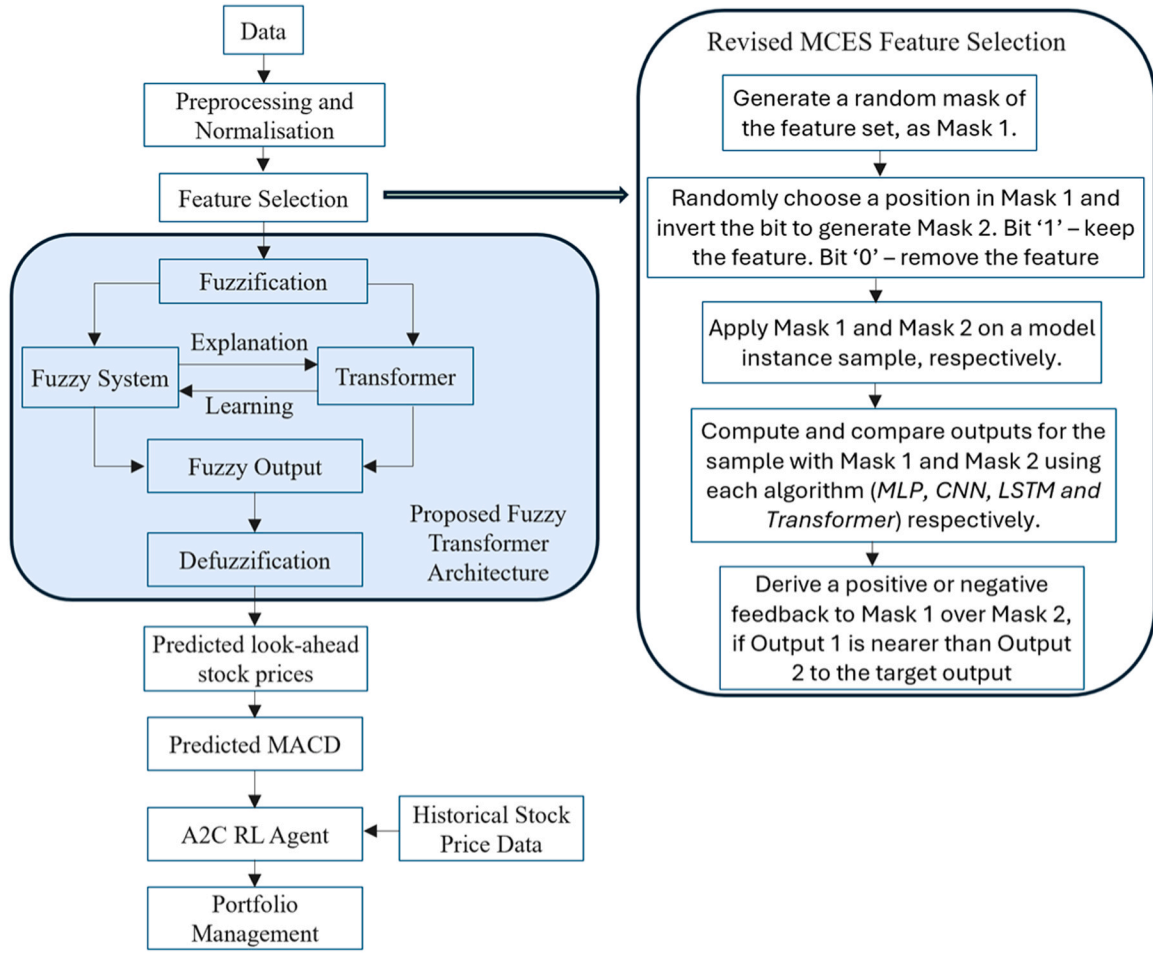


Fig. 2. Overall framework utilized the proposed fuzzy Transformer architecture.

shown in Fig. 3, the input layer, fuzzification layer, inference/rule base layer, consequent layer, and output layer. The input layer receives the input signals $\{X_1, X_2, \dots, X_j\}$, which are fed into the second layer, fuzzification layer.

3.4.1. Layer 2 - fuzzification layer

The nodes in this layer $\{c_{11}, c_{12}, \dots, c_{1n}, c_{21}, \dots, c_{ij}\}$ are the antecedents that constitute the fuzzy rule $\{R_1, R_2, \dots, R_n\}$. Each input X_i is turned into multiple clusters derived from the DBSCAN clustering algorithm. Each cluster is then turned into fuzzified output according to the Gaussian membership functions. The process is computed as shown in Eq. (5).

$$f_{ij} = \text{DBSCAN}(X_i)$$

$$O_{ij} = e^{-\frac{(f_{ij} - \mu_{ij})^2}{2\sigma_{ij}^2}} \quad (5)$$

where X_i is the i^{th} input feature; j denotes the j^{th} cluster; $\text{DBSCAN}()$ is the function that changes the inputs to clusters; μ_{ij} is the mean from the cluster c_{ij} ; σ_{ij} is the standard deviation of the cluster c_{ij} ; and O_{ij} denotes the derived output of this layer.

All the membership values obtained after passing through this layer are linked together, which will be the inputs to the third layer. For each data, the dimension relationship between the input shape and output shape of this layer is shown in Eq. (6).

$$\text{Input shape dimension} = (\text{Number of features}, 1)$$

$$\text{Output shape dimension} = (\text{Number of features}, \text{Max}(|\text{cluster } X_i|)) \quad (6)$$

where $\text{Max}(|\text{cluster } X_i|)$ denotes the maximum number of clusters among the features.

3.4.2. Layer 3 - inference/rule base layer

There are two paths in this layer: the fuzzy logic path and the Transformer path. In Layer 3, all learning is carried out by Transformer, while the fuzzy logic is responsible for generating the rules and assigning a weight to each rule. Therefore, the training and inference of Transformer behave much like in a standard deep-learning framework, with the addition of encoder and decoder modules. The fuzzified input clusters produced in Layer 2 serve as inputs to Transformer.

In this way, raw data are transformed into higher-dimensional representations, expressed as clustered membership values. They encode global information (i.e., the clusters) into each input feature, making the inference process more accurate. On the other hand, thanks to this more efficient and expressive model learning of fuzzy Transformer, the fuzzy rules can better capture the underlying patterns in the data. Moreover, because rule generations do not backpropagate gradients into Transformer, it does not impair the model's performance. Another advantage is that the attention weights of Transformer can offer additional insights into the generated rules, helping interpret the relationships within the data. This aspect will be discussed in more detail in Sub-Section 3.5.

Mamdani fuzzy rules are used in the fuzzy logic path, where the antecedents from the fuzzification layer are connected to the consequents in this layer, as shown as follows:

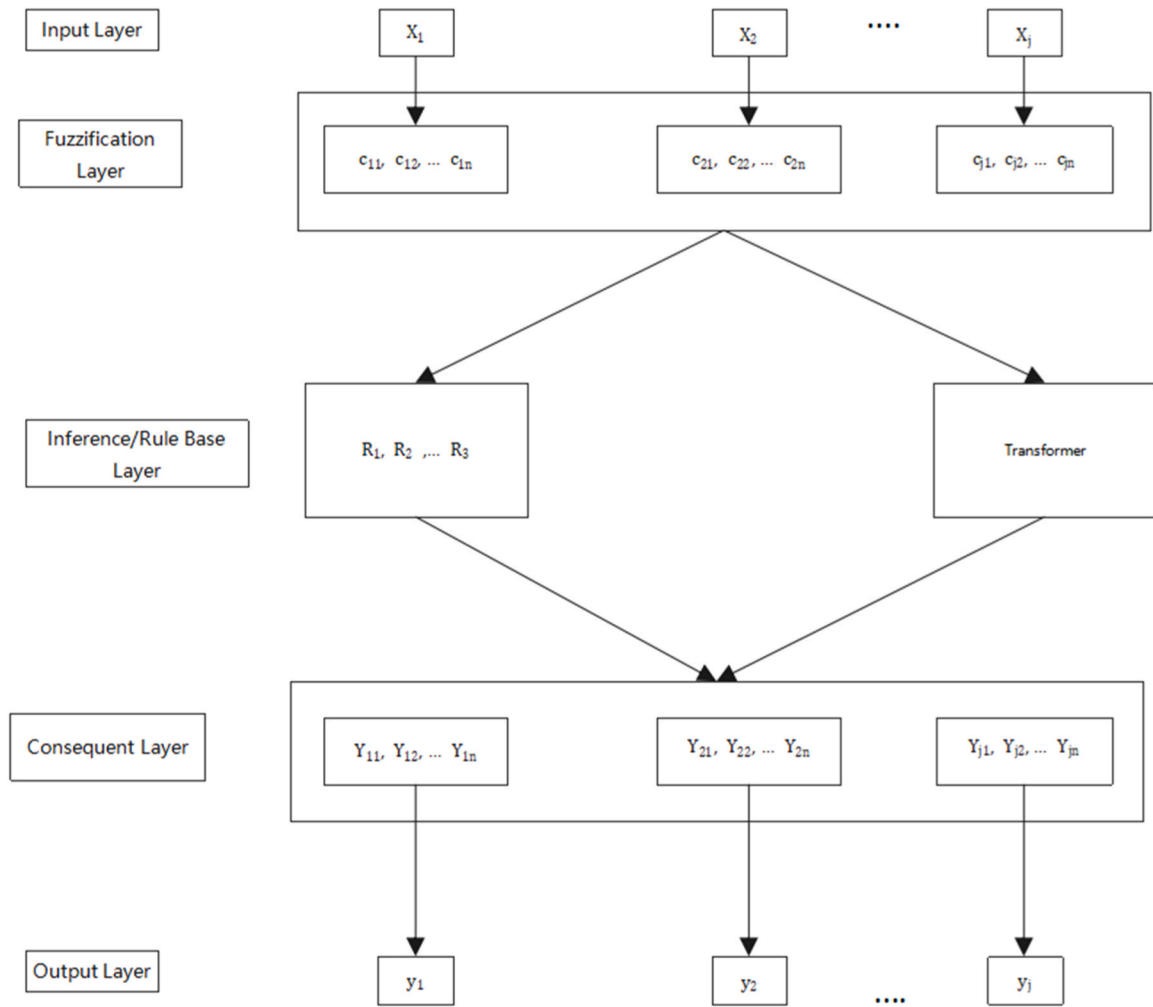


Fig. 3. The fuzzy Transformer architecture.

IF X_1 is c_{1a}^k and X_2 is c_{2b}^k and ... X_n is c_{nz}^k , THEN y_1 is Y_{1p}^k and ... y_n is Y_{mq}^k

where $\{c_{1a}^k, c_{2b}^k, \dots, c_{nz}^k\}$ denote the winning clusters for the fuzzified inputs (i.e., the antecedent), connected to the fuzzy rule k . $\{Y_{1p}^k, \dots, Y_{mq}^k\}$ denote the winning clusters for the outputs (i.e., the consequent), connected to the fuzzy rule k . There will be some common rules or rarely seen rules in this layer. As such, a weight is assigned to each rule to represent the importance of each fuzzy rule. The greater the weight is, the greater the importance is.

The Transformer model adopts an encoder-decoder structure [57]. Both the encoder and decoder comprise a stack of layers. Each encoder layer consists of two sublayers: multi-head attention and positionwise feed-forward network (FFN) [58]. Its attention mechanism allows the model to focus on the most dominant or relevant tokens in a sequence [21]. In order to learn the different tendencies, multi-head attention is usually used. The same MLP is employed for FFN transforming the representations at all positions in the sequence. Each sublayer adopts a residual connection, which is added with a position of the input sequence. The addition operation (Add) is followed by a layer normalization (Norm). The output of the encoder is a vector representing every position in the input sequence. The encoder output vector goes to the sub-layers of the decoder. The output of each step is fed to the bottom decoder in the next time step. Each decoder layer consists of three sublayers: multi-head attention, encoder-decoder attention, and positionwise FFN. Transformer models are known to be capable of capturing

long-range dependencies and parallel processing sequence data. It is reported to extend in time series data analytics, although certain drawbacks are reported such as black-box nature and high computational complexity.

In the proposed fuzzy Transformer, there are four layers in the Transformer architecture, as shown in Table 1. The first layer is a multi-layer Transformer encoder, followed by an MLP layer, dropout layer and another MLP layer. The Multi-layer Transformer encoder is modified with the specific model which consists of various steps in Fig. 4. The input data goes through the layer normalization (Norm), followed by the multi-head attention and dropout steps. Next, the addition with the residual connection is performed, that is followed by the layer normalization again. Two steps of 1-Dimensional Convolution (Conv1D) are conducted, with the dropout in between. The addition operation with the residual connection is performed next.

The Transformer encoder does not change the shape of the data. The dimensions of the input shape and output shape are shown in Table 2 among the Transformer encoder, Attention matrix, and Transformer as a

Table 1
Layers in the Transformer part.

Layer in Transformer	Module
1	Multi-layer Transformer encoder
2	MLP
3	Dropout
4	MLP

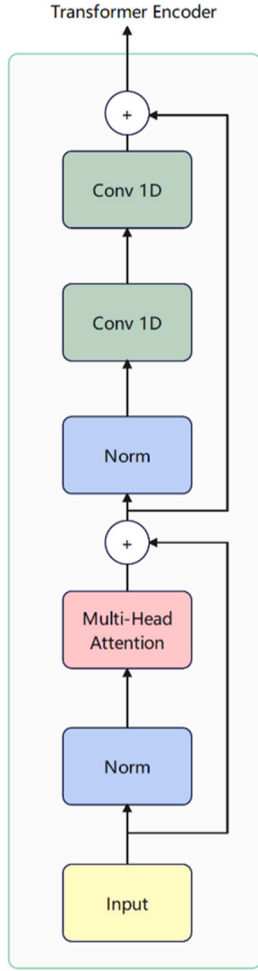


Fig. 4. Adopted multi-layer Transformer encoder.

Table 2

Dimensions of the input shape and output shape.

	Input shape dimension	Output shape dimension
Transformer encoder	(Number of features, $\text{Max}(\text{cluster } X_i)$)	(Number of features, $\text{Max}(\text{cluster } X_i)$)
Attention matrix	(Number of features, $\text{Max}(\text{cluster } X_i)$)	((Number of features, $\text{Max}(\text{cluster } X_i)$), (Number of features, $\text{Max}(\text{cluster } X_i)$))
Transformer as a whole	(Number of features, $\text{Max}(\text{cluster } X_i)$)	(Number of output Y, $\text{Max}(\text{cluster } Y_i)$)

whole. Three of them have the same dimension of input shapes as (Number of features, $\text{Max}(|\text{cluster } X_i|)$). The output shape of Attention matrix has a higher dimension as: (Input shape of the Transformer encoder) $\times$ (Input shape of the Transformer encoder).

3.4.3. Layer 4 - consequent layer

The data in this layer are membership values based on the fuzzification training data, similar to those of the fuzzification layer. Transformer is trained on these values and tries to fit the data. The input of this layer is the output from the Transformer in the third layer. As such, the input shape of this layer is same as the output shape of Transformer.

3.4.4. Layer 5 - output layer

The main task of this layer is to defuzzify the fuzzy outputs in the 4th layer into crisp outputs. The defuzzification calculation used is the

Center of Area method shown in Eq. (7).

$$Y_i = \frac{\sum_{j=1}^n m_{ij} \times \mu_{ij} \times \sigma_{ij}}{\sum_{j=1}^n m_{ij} \times \sigma_{ij}} \quad (7)$$

where Y_i denotes the i^{th} crisp output; m_{ij} denotes the membership value at Y_{ij} ; μ_{ij} is the mean value derived from the cluster Y_{ij} ; and σ_{ij} is the standard deviation value obtained from the cluster Y_{ij} . The dimensions of the input shape and output shape for this layer are shown in Eq. (8).

Input shape dimension = (number of output Y, $\text{Max}(|\text{cluster } Y_i|)$)

Output shape dimension = (number of output Y, 1) (8)

3.5. Rule formation

This sub-section explains how to filter out some rules among all the rules generated. Each fuzzy rule is specified as $\{A \rightarrow B\}$, where A denotes the antecedent and B denotes the consequent, that are composed of the winning clusters. The winning clusters refer to the clusters with the highest membership value among the multiple clusters generated from the data points. The weight of each rule is derived using Eq. (9).

$$W = \min_i(\mu_{A_i}(X_i)) \times \mu_B(Y_i) \quad (9)$$

where $\mu_{A_i}(X_i)$ is the membership value in the winning cluster belonging to X_i ; and $\mu_B(Y_i)$ is the membership value in the winning cluster belonging to Y_i .

During the generation of each rule, if the same rules are found, their weights are added. At the end of each iteration, if an existing rule does not appear in this iteration, the weight of this rule is reduced by multiplying a forgetting factor λ ($\lambda < 1$). It is to reduce the importance of the existing rules that are not in frequent usage. In the financial market, rules change dynamically all the time. It is necessary to appropriately reduce the number of rules gradually. The value of λ is calculated using Eq. (10).

$$\lambda = e^{-\frac{|p-l|}{\sqrt{n_i \times n_r}} + 1} \quad (10)$$

where $(p-l)$ denotes the difference between the current iteration and the last occurrence of this rule, indicating how long this rule has been absent; n_i denotes the number of times this rule appears; and n_r denotes the number of all rules in the current iteration. In order not to multiply a too small forgetting factor in weight, leading to unstable results for the rule formation, we use the segmentation function [10] to limit the range in Eq. (11).

$$\lambda = \begin{cases} 0.9, & \lambda < 0.9 \\ 0.99, & \lambda > 0.99 \\ \lambda, & \text{otherwise} \end{cases} \quad (11)$$

Next, the rule-node tagging is performed by setting a threshold for each value after the activation function in the MLP layer within Transformer. It examines which nodes are activated during the generation process of each rule.

The importance of rules is indicated with the rule weights. Several examples of generated rules are shown in Table 3. With the forgetting factor, the weights indicate how frequently each rule occurs. The rule weight is affected by the number of occurrences, with higher weights for more occurrences. But this way may pose potential problems. In certain situations, the rule weights for the extreme cases (e.g., input features with *Extremely High* or *Extremely Low* antecedents) are usually small, due

Table 3Examples of generated rules at look-ahead $t + 1$ day.

Rule	Weights	Input Features					Output Y
		Dc(9)	Dc(3)	Dc(7)	Dc(4)	Dc(5)	Dc(1)
R1	1.7829	Low	Normal	Low	Normal	Normal	Low
R2	2.8926	Low	Low	Normal	Normal	Normal	Low
R3	0.38	Normal	Extremely High	Very High	High	Normal	Extremely High

to the less occurrences of huge price changes in the financial markets. It has higher risks in such extreme cases, but it can bring much higher returns potentially. Some risk-taking investors may still do the buy or sell trading. As such, we should not determine the importance of a rule only depending on its frequency of occurrence. We should also compute the importance of each feature in each rule based on the attention layer. The procedure to derive the weights of rule importance is as follows.

- When generating each rule, the data and the winning cluster which is used to generate this rule are saved together.
- The attention matrix is multiplied with this saved data to get the attention coefficient matrix for each cluster.
- Based on the saved winning cluster information, the rule importance weight for this cluster is derived.

With this procedure, we use 4 multi-head attentions to derive the weight of each input feature. A few examples are shown in Table 4. It is observed that the derived weights or the degree of attention to each feature are relatively similar, indicating that these features are useful. Linking up the antecedent values of the features in Table 3 and the derived attention scores in Table 4, it is seen that the features with the Low antecedent of rules are more important according to the attention scores. It means that more attentions will be paid to these features when deriving the final output from the fuzzy Transformer architecture.

3.6. Portfolio management with the predicted MACD indicator and RL

After incorporating the predicted n -days look-ahead stock prices, the predicted MACD is utilized by the A2C RL. In A2C RL, Actor is policy based as responsible for decision making. Critic is value based as responsible for evaluating current Actor. The Advantage function is derived to evaluate how much better than average the chosen action a is in the state s at time t under policy π , as shown in Eq. (12).

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) \quad (12)$$

If both values of $A^\pi(s_t, a_t)$ and $Q^\pi(s_t, a_t)$ need to be predicted, it may lead to a worse loss. As such, the A2C RL only predicts $V^\pi(s_t)$, that is next used to represent $Q^\pi(s_t, a_t)$. Assuming accurate estimates can be made, the Q function can be represented as in Eq. (13).

$$Q^\pi(s_t, a_t) = r_t + V^\pi(s_{t+1}) \quad (13)$$

Then, the advantage function is revised to be shown in Eq. (14).

$$A^\pi(s_t, a_t) = r_t + V^\pi(s_{t+1}) - V^\pi(s_t) \quad (14)$$

Using only MACD as the input is insufficient due to the scant data and the inability to predict future prices accurately at each moment. It prevents training a good RL agent. Therefore, we concatenate the predicted MACD value with the historical stock price information. As such,

Table 4Examples of derived weights of each feature at look-ahead $t + 1$ day.

Rule	Derived weights for each input feature					Output Y
	Dc(9)	Dc(3)	Dc(7)	Dc(4)	Dc(5)	Dc(1)
R1	1.6118	1.4961	1.6274	1.5980	1.4471	Low
R2	1.6235	1.6978	1.5021	1.5029	1.4674	Low

the input vector of A2C RL to be utilized consists of seven dimensions as: (*Open price, High price, Low price, Close price, Previous close price, Transaction volume, Predicted MACD*). The input vector after pre-processing and feature selections will be fed into the proposed fuzzy Transformer architecture. The configurations of the A2C RL agent are as follows.

State: We define the state as: $S_t = \{F_{i,t}, W_{i,t}\}$, where $F_{i,t}$ denotes the features of the i^{th} stock at time t , and $W_{i,t}$ denotes the weight of the i^{th} stock in the portfolio at time t . For the i^{th} stock, the value of $F_{i,t}$ is computed in Eq. (15).

$$F_{i,t} = \{f_{i,t-n}, f_{i,t-n+1}, \dots, f_{i,t}\} \quad (15)$$

$$f_{i,t-n} = \{P_{i,t-n}^{open}, P_{i,t-n}^{high}, P_{i,t-n}^{low}, P_{i,t-n}^{close}, P_{i,t-n}^{previous\ close}, Volume_{i,t-n}, MACD_{i,t-n}\}$$

where P denotes the price of the stocks in the portfolio.

Action: We re-assign weights to each stock in the portfolio based on the decisions of the A2C RL agent in each state. The final action is the result after normalization through the SoftMax function as shown in Eq. (16).

$$a_t = W_{t+1}$$

$$W_t = \text{Softmax}(a_t) \quad (16)$$

We discovered that the weights derived from the SoftMax function can be interpreted as the likelihood of each stock having the greatest upward movement. This led us to wonder whether artificially increasing the weight of the stock would yield better results, that the RL agent perceives to have the highest growth potential. From a risk minimization perspective, it might not be a good idea, as an overly large weight for a particular stock might increase risk in the portfolio. But from the opposite point of view, this approach can invest most of the money in a stock with a strong upward trend to maximize profits. Therefore, we modify Eq. (16) using a manually specified parameter to try to consider both perspectives. Our changes for Action are shown in Eq. (17).

$$a_{t \arg \max(a_t)} \leftarrow a_{t \arg \max(a_t)} + \delta e^{\max(a_t)} \quad (17)$$

$$W_t = \text{Softmax}(a_t)$$

where $a_{t \arg \max(a_t)}$ denotes the element of a_t that has the largest value; $\max(a_t)$ is the largest value of a_t ; δ is a hyper parameter, controlling the extent to which weight is assigned primarily to the best action; and W_t denotes the weight of the stock. In the experiments of this work, the value of δ is set as 1, i.e., $\delta = 1$.

Reward: We define the total price change as the reward of the RL agent, shown in Eq. (18) and Eq. (19).

$$g_t = \frac{\sum_{i=1}^k P_{i,t}^{close} \times W_{i,t}}{\sum_{i=1}^k P_{i,t-1}^{close} \times W_{i,t-1}} \quad (18)$$

$$r_t = V_{t-1} \times g_t - C \quad (19)$$

where r_t denotes the reward function of the i^{th} step; V_0 is the initial state value; V_t denotes the state value of the i^{th} step; and C denotes the commission fee per stock trading transaction. In order to obtain higher

profits, we modify the reward function r_t in Eq. (19) by adding a penalty term to force the RL agent to allocate more money into a particular stock. The modified reward function r_t is computed in Eq. (20).

$$r_t = V_{t-1} \times g_t - C + \mu \text{Var}(a_t) \quad (20)$$

where $\text{Var}(a_t)$ is variance of a_t ; and μ is a hyper parameter. In the experiments of this work, the value of μ is set as 150.

3.7. Evaluation metrics

After the denormalization of the defuzzification result, we can get the predicted delta change result. With it, the predicted stock price can be calculated accordingly. We are using a multi-model system, so number of m models are needed to predict the results for the next n days. The specific formula from denormalization to get the final prediction result is shown in Eq. (21).

$$O_{ij} = |x_i|_2 \times y_{ij} + x_{ij} \quad (21)$$

where O_{ij} denotes the j^{th} value in the i^{th} window after denormalization; $|x_i|_2$ denotes the L2 norm of the array x_i ; y_{ij} denotes the j^{th} value in the i^{th} window coming to the output of the model; and x_{ij} denotes the j^{th} value in the i^{th} window before the normalization.

The predicted look-ahead stock price with multiple models is calculated at Eq. (22).

$$\hat{p}(t+n) = p(t) + \text{model}_1(p(t+1)) + \text{model}_2(p(t+2)) + \dots + \text{model}_m(p(t+n)) \quad (22)$$

where $\hat{p}(t+n)$ denotes the predicted n days look-ahead price at time t ; $p(t)$ is the actual price at time t ; $\text{model}_m(p(t+n))$ is predicted n days look-ahead delta change by the fuzzy Transformer architecture. It is observed from Eq. (22) that our model is an error accumulation model and may not make effective look-ahead predictions for a large number of days. The performance of our model is evaluated using the RMSE, R^2 score, MAPE, and MAE, shown in Eq. (23).

$$\begin{aligned} RMSE &= \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \\ R^2 &= 1 - \frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2}{\sum_{i=1}^N (y_i - \bar{y}_i)^2} \\ MAPE &= \frac{1}{N} \times \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \\ MAE &= \frac{\sum_{i=1}^N |y_i - \hat{y}_i|}{N} \end{aligned} \quad (23)$$

where y_i is the actual value; $\hat{y}_i$ is the predicted value; $\bar{y}_i$ is the mean value of y_i .

3.8. Interpretability of the fuzzy transformer architecture

It is observed in Eq. (22), it needs m models to predict the look-ahead prices $\hat{p}(t+n)$ for the next n days. It means that m sets of features are needed. If these m sets of features are highly overlapped, it will be beneficial to use the frequently recurring features as the only inputs to train the m models. If we directly use these highly overlapped features as the inputs, it can omit the MCES feature selection process and the clustering of the input data, greatly reducing the computational

complexity of the model. The number of rule combinations appearing in the antecedent part is $|\text{cluster}|^{\text{Feature}}$, which denotes the number of clusters by the power of the number of features. If there are too many rules in the system, it is not in favor for the sake of interpretability. For the aspects of the interpretability for the proposed fuzzy Transformer architecture, we can distinguish which rule is more important based on the frequent appearances and the weight of each rule. The fuzzified input data is fed into Transformer. In this context, the attention score in Transformer indicates the model degree of focus on a particular feature of input. As such, the attention scores in the fuzzy Transformer architecture can be used to assess which elements in the rule antecedent play a more important role.

To facilitate understanding of the weight of each antecedent in the generated rule, we show an illustrative visualization example in Fig. 5. In fuzzy rule nodes section, i.e., the upper part of Fig. 5, the antecedents and consequents are connected by the red line corresponding to a fuzzy rule extracted from the data. The antecedents in this example are c_3 from Feature 1, c_1 from Feature 2, and c_5 from Feature 3. Then, in the first attention matrix of the Transformer path, i.e., the bottom part of Fig. 5, we extract only the attention weights for those antecedents used in the generated rule. Concretely, we retrieve the weights at the indices corresponding to these antecedents from the output of the multi-head attention layer and compute their mean values across all heads, since we employ multi-head attention. Traditional rule weights can tell us which rules are important, but they cannot explain which antecedents within those rules play the key role. By combining fuzzy logic with Transformer, we overcome this limitation and further enhance interpretability.

Therefore, by combining the feature selection, attention layer of Transformer, and fuzzy rules, the proposed fuzzy Transformer architecture can achieve good interpretability. The good interpretability addresses the black-box nature of the neural network architecture. It enables users to analyze and understand easily on these components: (i) which features in the data are important, (ii) which fuzzy rules are important in the system, (iii) which elements of the rule antecedent are in what state, e.g., small, large, etc., (iv) which element plays a key role in constructing this fuzzy rule, and (v) when different antecedents lead to the same consequence, the more important elements of this rule can be further distinguished based on the attention score. These components help us understand the entire FNN system and its underlying rules much better.

3.9. Training and inference speed

We used Yahoo Finance to collect historical market data. The proposed fuzzy Transformer architecture was implemented with Python programming language and packages. Data loading and preprocessing were relied on NumPy, Pandas, and Scikit-learn Python packages. All neural network models were built in TensorFlow. Reinforcement Learning experiments were leveraged FinRL, a deep RL trading library [59]. Training curves and performance metrics were plotted and visualized with Matplotlib Python package.

On our experiment platform with Intel 8th-Gen i7 CPU and NVIDIA GTX 1060 GPU, the proposed fuzzy Transformer requires on average 3.74 s per training epoch, and 1.15 s to generate fuzzy rules. During inference, the Transformer forward pass itself takes 0.11 s only; but the defuzzification step dominates runtime at 87 s per batch. This shows that, while the neural network components are efficient, the fuzzy logic layer currently constitutes a computational bottleneck. Given the current defuzzification latency of 87 s, the end-to-end system cannot produce signals fast enough for true real-time high frequency trading at sub-second or intraday trading. To reduce the speed gap to real-time use, future work is needed to prune or approximate the fuzzy rule base to shrink defuzzification time, and parallelize the inference engine across better GPU or CPU.

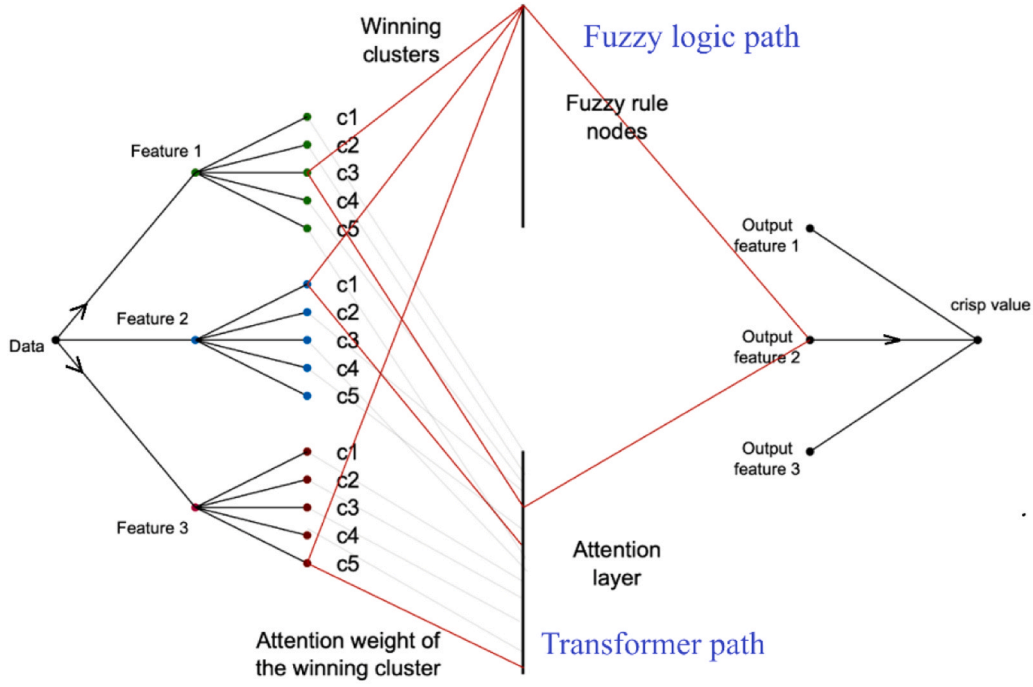


Fig. 5. A visualization example of interpretability of the fuzzy Transformer architecture.

In our experiments, we decouple the internal time step of the model from the portfolio holding period. We perform the rebalancing of portfolio once per calendar day, regardless of the sampling rate of Transformer. To limit transaction commission costs, we also enforce a minimum threshold on weight changes, where small shifts smaller than the threshold are skipped without performing buy or sell transactions.

4. Experiment on portfolio rebalancing

SPDR Dow Jones Industrial Average ETF Trust (DIA) and SPDR S&P 500 ETF Trust (SPY) are two ETF trusts in the financial market which are widely used to track economic trends in U.S. stock market. DIA tracks the Dow Jones Industrial Average index consisting of 30 well-established and financially sound large companies. SPY is designed to track the Standard & Poor's (S&P) 500 index, which reflects the stock prices of 500 large companies in U.S. stock markets. The stock of Apple Inc (AAPL) whose market capitalization is about \$2.6 trillion is one of the most widely traded stocks in U.S. market. These three assets are selected to construct a financial portfolio. The performance of the fuzzy Transformer architecture FNN system is evaluated with the experiment on this portfolio in this section.

For these three assets, i.e., DIA, SPY, and AAPL, we have chosen their time series price data in the period from December 2017 to January 2023. Among them, the price data from Dec. 2017 to Dec. 2020 are used for training, and the price data during Jan. 2021 to Jan. 2023 serve as the test data.

4.1. Results of feature selection

The price delta change (Dc) and momentum (Mt) of each financial asset are the candidates for the feature selections. With the MCES feature selection method used, the highest weighted features can be identified and selected for the FNN system. Four types of neural networks are employed: MLP, CNN, LSTM and Transformer that gives feedback to MCES, to derive feature selection results. The feedback from these four types of networks is then normalized and summarized to derive the finalized results of feature selection for the model in the combination of each asset and each look-ahead day of predictions (i.e., $t + 1$, $t + 2$, ...,

$t + 10$). A number of five rounds of experiments are conducted for each model. Some examples of the feature selection for the models of SPY at $t + 2$, $t + 8$, and $t + 10$ look-ahead days are shown in Table 5. The features with the highest number of appearances in all rounds are selected as the important features for each model. With this approach, the 10 models of SPY at $t + 1$ to $t + 10$ look-ahead predictions for Transformer are consolidated in Table 6.

It is observed from Table 6 that the velocity-based features appear 35 times and the momentum-based features appear 15 times. For the velocity-based features, the Dc(7) feature appears the most frequently, showing up 8 times. Next is the Dc(3) feature, which appears 6 times. Following that are Dc(9) and Dc(8), appearing 5 and 4 times, respectively. For the momentum-based features, the highest number of appearances occur by the Mt(7), Mt(5), and Mt(6) features, each appearing 3 times.

Two observations are obtained through the experiments on SPY: (i) although the momentum-based feature appears less frequently, it still becomes one of the top five features, indicating that this kind of feature is still helpful for predicting delta change of prices; (ii) there are indeed some frequently occurring features. Out of a total of 20 features, the 6 most frequently appearing features account for 64% of occurrences.

Table 5

Examples of feature selection results for SPY at various look-ahead days.

Method	1st round	2nd	3rd	4th	5th
Model: SPY at look-ahead days = $t + 2$					
MLP	Dc(7)	Dc(3)	Mt(7)	Mt(3)	Dc(4)
CNN	Mt(6)	Mt(7)	Mt(8)	Dc(6)	Dc(3)
LSTM	Dc(7)	Mt(7)	Dc(4)	Mt(2)	Dc(8)
Transformer	Dc(7)	Dc(8)	Dc(3)	Dc(2)	Dc(6)
Model: SPY at look-ahead days = $t + 8$					
MLP	Dc(7)	Dc(8)	Dc(3)	Dc(6)	Dc(9)
CNN	Mt(6)	Mt(5)	Mt(7)	Dc(7)	Mt(8)
LSTM	Dc(7)	Dc(8)	Dc(5)	Mt(6)	Dc(3)
Transformer	Dc(8)	Dc(6)	Dc(4)	Mt(6)	Mt(3)
Model: SPY at look-ahead days = $t + 10$					
MLP	Dc(7)	Dc(5)	Dc(9)	Dc(4)	Dc(3)
CNN	Mt(5)	Mt(6)	Dc(3)	Dc(7)	Dc(2)
LSTM	Dc(7)	Mt(1)	Mt(7)	Dc(5)	Mt(2)
Transformer	Dc(3)	Mt(3)	Dc(5)	Dc(7)	Dc(2)

Table 6Consolidated feature selection results for SPY from $t + 1$ to $t + 10$.

Look-ahead	1st round	2nd	3rd	4th	5th
$t + 1$	Dc(9)	Dc(3)	Dc(7)	Dc(4)	Dc(5)
$t + 2$	Dc(7)	Mt(7)	Dc(3)	Dc(8)	Dc(4)
$t + 3$	Dc(3)	Mt(7)	Dc(6)	Dc(1)	Dc(8)
$t + 4$	Dc(3)	Dc(8)	Mt(4)	Dc(7)	Mt(8)
$t + 5$	Mt(9)	Mt(7)	Dc(5)	Dc(3)	Dc(7)
$t + 6$	Dc(2)	Dc(8)	Dc(9)	Dc(4)	Dc(7)
$t + 7$	Dc(9)	Dc(1)	Mt(2)	Mt(5)	Dc(5)
$t + 8$	Dc(9)	Dc(7)	Dc(6)	Mt(6)	Mt(5)
$t + 9$	Mt(3)	Dc(7)	Mt(4)	Mt(6)	Dc(1)
$t + 10$	Dc(9)	Dc(7)	Mt(5)	Mt(6)	Dc(3)

Based on this discovery, the performance of the SPY models will be evaluated using only these features.

4.2. Results of price look-ahead predictions

With the dataset and identified features, the n -day look-ahead prices are predicted by the fuzzy Transformer architecture with the time step as 10 for each financial asset. The prediction results for the three assets: DIA, SPY and AAPL are shown in Table 7.

As seen in Table 7, the look-ahead prediction performances of the models become worse gradually as the number of look-ahead days increase due to the accumulation of errors. As the predicted stock prices will be employed to compute the predicted MACD values, the RMSE appears to be a bit more important. It is observed that on SPY, there is a relatively low RMSE value from $t + 1$ to $t + 5$ look-ahead prediction by the fuzzy Transformer architecture. But from $t + 6$ to $t + 10$ look-ahead prediction, the RMSE values increase at a large rate, ultimately reaching 72.5487. Therefore, when calculating the predicted MACD of SPY, we choose up to $t + 5$ as the look-ahead prediction ranges. The RMSE values obtained by the fuzzy Transformer architecture are shown side by side with the corresponding look-ahead predictions on SPX (S&P index) by IFDNN [10], FE-RNN [21], and MEMD-LSTM [60]. It is only for the reference purpose to help visualize how well the prediction of the fuzzy Transformer is. But it would not be considered as a fair comparison, as the prices of SPY ETF are not equal to those of SPX in the same period, although SPY ETF is derived from the prices of SPX.

On DIA, it is seen that the overall RMSE values are not very large from $t + 1$ to $t + 10$. But the R^2 scores reduce quickly from $t + 3$ onwards. It indicates the prediction results of the models becoming unreliable. Hence, when calculating the predicted MACD of DJIA, we choose up to $t + 2$ look-ahead as the prediction range. Similarly, The RMSE values on DIA obtained by the fuzzy Transformer are displayed side by side with the look-ahead predictions on DJIA Index by FE-RNN [21] in Table 7. The prices of DIA are derived from those of the DJIA index during the same period.

On the AAPL, combining RMSE and R^2 score, it is observed that the

look-ahead predictions are reliable up to $t + 5$ days. However, from $t + 8$ days onwards, the RMSE error sharply increases. The R^2 scores become a bit too low as well. As such, when calculating the predicted MACD of AAPL, we choose up to $t + 5$ look-ahead as the prediction range.

Further experiments are conducted to compare the performance of the proposed fuzzy Transformer architecture on $t + 1$ look-ahead prediction, with several other popular deep learning neural networks including Transformer, LSTM, Bi-LSTM, and RNN using the same dataset. The experiment results are shown in Table 8. It is observed that the proposed fuzzy Transformer architecture outperforms Transformer and RNN models. For DIA, the fuzzy Transformer architecture achieves similar performance to LSTM, while loses out to Bi-LSTM on RMSE, R^2 and MAE, but with 43.2% better on MAPE. For SPY, the proposed fuzzy Transformer architecture outperforms Bi-LSTM with 0.2–33.3%, while loses out to LSTM with -0.4% to -11.8% . For AAPL, the fuzzy Transformer architecture performs better than LSTM with 0.4–223.8%, while loses out to Bi-LSTM on RMSE, R^2 and MAE with -0.04% to -26.2% , but scores much better on MAE with 54.4%.

The average value of the improvements on DIA, SPY and AAPL with these four metrics RMSE, R^2 , MAPE and MAE is calculated by summarizing all 12 improvement values in each row, then divided by 12. It is observed that the proposed fuzzy Transformer architecture achieves higher average improvement results, outperforming Bi-LSTM by 3.2%, and outperforming LSTM by 25.8%.

4.3. Trading with the predicted MACD indicator

In this sub-section, the performance of the predicted MACD is compared with that of the vanilla MACD. The predicted MACD is derived by combining past historical prices and look-ahead forecasted prices. In the experiments, different look-ahead predicted prices for each financial asset are chosen by the feature selection phase for the fuzzy Transformer. As stated in Section 4.2, the DIA dataset chooses shorter look-ahead predicted prices at $t + 2$, while longer look-ahead predicted prices at $t + 5$ are chosen for the SPY and AAPL datasets. The performance of the predicted MACD by the fuzzy Transformer is compared with several benchmark deep learning neural networks in the experiments. The MACD prediction performance of these methods is shown in Table 9. It is observed MACD predicted by the proposed fuzzy Transformer architecture outperforms that of vanilla MACD and Transformer, with average improvement at 78.1% and 12.1%, respectively. MACD predicted by the fuzzy Transformer architecture also achieves better performance in most metrics than those predicted by LSTM and RNN, with average improvement at 2.8% and 10.3%, respectively. For DIA, MACD predicted by the fuzzy Transformer architecture loses out to that of Bi-LSTM with -3.1% to -26.7% . For SPY, MACD predicted by the fuzzy Transformer outperforms Bi-LSTM on R^2 and MAPE by about 17.0%, while it loses out to MACD predicted by Bi-LSTM on RMSE with -20.9% . Similar observations are seen for AAPL, where MACD

Table 7Predicted results look-ahead of $t + 1$ to $t + 10$.

Look-ahead	Fuzzy Transformer in this paper						IFDNN [10]		FE-RNN [21]		MEMD-LSTM [60]
	DIA		SPY		AAPL		SPX (S&P 500) Index		DJIA Index	SPX	SPX
	RMSE	R^2	RMSE	R^2	RMSE	R^2	RMSE	R^2	RMSE	RMSE	RMSE
$t + 1$	5.6119	0.8674	3.6185	0.9831	0.8618	0.9969	68.33	0.991	293.695	28.221	36.019
$t + 2$	8.0138	0.7286	5.1141	0.9662	4.4431	0.9189	73.28	0.990	399.906	34.900	
$t + 3$	14.5744	0.0986	7.5888	0.9255	4.0636	0.9322	72.43	0.990	529.295	46.875	38.568
$t + 4$	20.0354	-0.7101	10.3708	0.8609	5.2173	0.8883	73.08	0.990	578.594	52.170	
$t + 5$	26.3415	-1.9673	14.1884	0.7395	6.5990	0.8212	79.64	0.988	612.125	58.754	39.975
$t + 6$	29.7638	-2.8014	25.0083	0.1901	10.2684	0.5667	81.49	0.987	637.920	63.323	
$t + 7$	33.3627	-3.7950	35.2463	-0.6102	10.6661	0.5321	82.35	0.987	683.641	66.569	
$t + 8$	33.5683	-3.8737	43.2814	-1.4319	11.2682	0.4765	86.48	0.985			
$t + 9$	30.3244	-2.9928	58.1968	-3.4035	15.7418	-0.0242	85.17	0.986			
$t + 10$	26.7084	-2.1094	72.5487	-5.8484	18.2225	-0.3763	89.16	0.985			

Table 8
Comparisons of PREDICTION PERFORMANCE.

Algorithms	DIA				SPY				AAPL				Average
	RMSE	R ²	MAPE	MAE	RMSE	R ²	MAPE	MAE	RMSE	R ²	MAPE	MAE	
Proposed fuzzy Transformer	<u>5.6119</u>	0.8674	<u>0.0037</u>	<u>1.7849</u>	<u>3.6185</u>	<u>0.9831</u>	<u>0.0051</u>	<u>2.1211</u>	<u>0.8618</u>	<u>0.9969</u>	<u>0.0037</u>	<u>0.2781</u>	
Transformer	9.2132	0.6613	0.0074	2.3976	4.0241	0.9795	0.0073	2.9574	2.2109	0.9798	0.0113	1.6638	
LSTM	5.8996	<u>0.9042</u>	0.0031	1.9931	3.4198	0.9871	0.0045	2.0092	1.2702	0.9933	0.0061	0.9005	
Bi-LSTM	3.9141	0.9867	0.0053	1.1215	3.8101	0.9816	0.0068	2.7856	0.6362	0.9973	0.0029	<u>0.4293</u>	
RNN	5.9223	0.7959	0.0062	1.9401	6.5313	0.9459	0.0092	3.7428	4.2995	0.9238	0.0166	2.4132	
Improvement of fuzzy Transformer from Transformer	61.2%	23.8%	100.0%	34.3%	11.2%	0.4%	43.1%	39.4%	156.5%	1.7%	205.4%	498.3%	97.9%
Improvement of fuzzy Transformer from LSTM	5.1%	−4.2%	−16.2%	11.7%	−5.5%	−0.4%	−11.8%	−5.3%	47.4%	0.4%	64.9%	223.8%	25.8%
Improvement of fuzzy Transformer from Bi-LSTM	−30.3%	−13.8%	43.2%	−37.2%	5.3%	0.2%	33.3%	31.3%	−26.2%	−0.04%	−21.6%	54.4%	3.2%
Improvement of fuzzy Transformer from RNN	5.5%	8.2%	67.6%	8.7%	80.5%	3.8%	80.4%	76.5%	398.9%	7.3%	348.6%	767.7%	154.5%

Table 9
Performance Comparisons of the Predicted MACD and vanilla MACD.

Method	DIA			SPY			AAPL			Average
	RMSE	R ²	MAPE	RMSE	R ²	MAPE	RMSE	R ²	MAPE	
Vanilla MACD	5.8043	0.3553	2.5988	7.7929	0.3103	1.8833	4.5387	0.3360	25.1308	
Predicted MACD by proposed fuzzy Transformer	4.5244	<u>0.5623</u>	2.2637	5.2761	0.6838	1.3718	3.0354	0.7030	5.2169	
Predicted MACD by Transformer	4.9541	0.4904	2.2788	6.2011	0.5622	1.6153	3.1632	0.5601	5.6410	
Predicted MACD by LSTM	4.4154	0.5548	<u>2.1885</u>	4.0897	<u>0.5851</u>	<u>1.4916</u>	3.1314	0.5728	5.6081	
Predicted MACD by Bi-LSTM	3.3173	0.5800	2.1659	<u>4.1727</u>	0.5681	1.6054	2.1201	<u>0.5773</u>	<u>5.4067</u>	
Predicted MACD by RNN	4.3504	0.4716	2.3985	4.3557	0.5293	1.7081	<u>2.2649</u>	0.5176	7.5148	
MACD improvement of fuzzy Transformer from vanilla MACD	28.3%	36.8%	14.8%	47.7%	54.6%	37.3%	49.5%	52.2%	381.7%	78.1%
MACD improvement of fuzzy Transformer from Transformer	9.5%	12.8%	0.7%	17.5%	17.8%	17.8%	4.2%	20.3%	8.1%	12.1%
MACD improvement of fuzzy Transformer from LSTM	−2.5%	1.3%	−3.3%	−22.5%	14.4%	8.7%	3.2%	18.5%	7.5%	2.8%
MACD improvement of fuzzy Transformer from Bi-LSTM	−26.7%	−3.1%	−4.3%	−20.9%	16.9%	17.0%	−30.2%	17.9%	3.6%	−3.3%
MACD improvement of fuzzy Transformer from RNN	−3.8%	16.1%	6.0%	−17.4%	22.6%	24.5%	−25.4%	26.4%	44.0%	10.3%

predicted by the fuzzy Transformer outperforms Bi-LSTM on R² and MAPE by 17.9% and 3.6%, respectively. While it loses out to that of Bi-LSTM on RMSE with −30.2%, the proposed fuzzy Transformer has better interpretability than Bi-LSTM.

There are various noises in deriving the predicted MACD values, which cause continuous fluctuations across the X-axis. It triggers many unnecessary buy and sell signals, that bring in excess costs on the transaction commissions. It leads to potential reductions in trading

profits. To address this issue, we constructed an auxiliary network that only makes buying or selling decisions under stable conditions. The specific steps are as follows:

- A label is assigned to the MACD value: ‘1’ is assigned if the MACD value is positive, and ‘0’ is assigned if it is negative.
- A model is trained that predicts the variance of these labels over the next n look-ahead days.

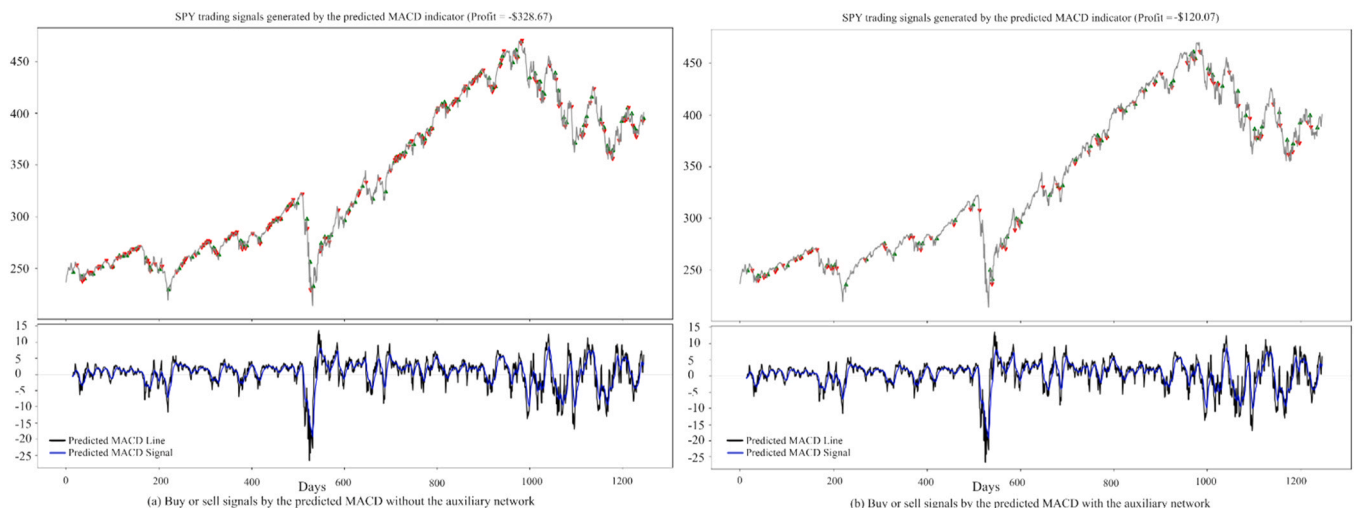


Fig. 6. Effects using the auxiliary network to reduce unnecessary transactions.

- A threshold value is defined, which allows the model to make buying or selling decisions only when its predicted MACD result is less than the threshold.

An example effect of the auxiliary network on SPY is given in Fig. 6 (a) without the auxiliary network, and Fig. 6(b) with the auxiliary network. The green dots denote the buying signals, and the red dots denote the selling signals generated by the predicted MACD indicators. Compared to the upper half figures of Figs. 6(a) and 6(b), the number of green dots and red dots are reduced significantly when using the auxiliary network in Fig. 6(b). It can reduce the unnecessary transactions and commissions. The use of the auxiliary network allows the fuzzy Transformer architecture to better deal with the concept drift, which serves as an excellent safeguard to enhance the stability in performance of the models.

4.4. Portfolio rebalancing by RL agent with predicted MACD

Next, the proposed system is employed to perform the portfolio rebalancing experiment with the predicted MACD indicator using RL. To ensure the training accuracy for the fuzzy Transformer architecture, the dataset with longer time frames of the three financial assets, DIA, SPY and AAPL, is selected, as shown in Table 10.

The cumulative return is chosen as the evaluation metric of the portfolio rebalancing in the experiment. The cumulative return is derived from the difference between the initial net asset value (NAV) and the final NAV, that is then normalised by the initial NAV of the portfolio. In the experiments, the A2C RL agent is adopted to conduct the portfolio rebalancing under different settings as follows:

- The input vector without MACD: (open price, high price, low price, close price, previous close price, transaction volume).
- The input vector with MACD: (open price, high price, low price, close price, previous close price, transaction volume, predicted MACD).
- The vanilla Action of the RL agent as shown in Eq. (16).
- The revised Action as shown in Eq. (17).
- The vanilla Reward of the RL agent as shown in Eq. (19).
- The revised Reward as shown in Eq. (20).

The portfolio cumulative returns of these different settings of the RL agent are compared in Table 11. It is observed that the cumulative returns are generally higher when incorporating MACD in the input vector. It indicates that MACD helps the RL agent make better decisions and is beneficial for performance improvements. The best performance is obtained by revising the Action of the RL agent, with the vanilla Reward, where 62.92% of the cumulative return achieved without MACD in the input vector, and 94.72% of return obtained with MACD in the input vector. When revising the Action, we generally lean towards stocks that the RL agent perceives to have significant upward potential. The RL agent can increase the weight of the stock with the highest rise potential in the portfolio. When incorporating MACD in the input vector, the RL agent can make more accurate judgments on up-trend stocks.

For the RL agent with MACD in the input vector under the vanilla Reward, the comparisons of the cumulative returns between the vanilla Action and revised Action can be visualized in Fig. 7.

It is seen in Fig. 7 that the portfolio NAV rebalanced by the RL agent with the vanilla Action tends to be closer to the NAV of DIA and SPY. The cumulative returns of the portfolio are slightly higher than those of DIA

Table 11

Cumulative return comparisons of different RL settings.

	Vanilla Action and Reward	Revised Action and Vanilla Reward	Vallina Action and Revised Reward	Revised Action and Reward
Input vector without MACD	49.84%	62.92%	49.07%	47.36%
Input vector with MACD	50.21%	94.72%	51.38%	45.22%
Return increment	0.37%	31.80%	2.31%	−2.14%

and SPY. This conservative strategy results in the cumulative return curve remaining at a relatively low level overall. In contrast, after revising the Action, the RL agent invests more money to AAPL during the rapid price rise of the AAPL stock. It causes the cumulative return to rise sharply by riding the increasing price trend of AAPL. However, when the price of AAPL drops sharply, the RL agent chooses to sell off AAPL and switch to the other two assets. It reduces the impact of the price decline of AAPL.

In the revised RL Action function as shown in Eq. (17), the hyper parameter δ is crucial to portfolio rebalancing performance. It controls the extent to which weight is assigned primarily to the best action. We conduct further experiments to better highlight its effect, with results shown in Fig. 8 under different values of δ . When δ is extremely large, each weight adjustment simply picks the single best asset and allocates all the weight to it, setting every other asset's weight to zero. As such, portfolio management effectively becomes a pure prediction problem. Conversely, when $\delta = 0$, the RL agent must accurately rebalance every weight, which turns it into a conventional reinforcement-learning task. Depending on the dataset and the specific objectives, the relative difficulty of prediction versus weight adjustment can vary. By tuning the value of δ , we strike a balance between these two problems, lowering the overall difficulty while leveraging the strengths of both supervised learning and reinforcement learning to yield a superior strategy. Essentially, our revised RL algorithm trades off between supervised learning and reinforcement learning based on the value of δ .

It is observed in Fig. 8 that performance peaks at $\delta = 5$. This indicates that, at this setting, the RL algorithm successfully combines predicting the best asset with rebalancing weights to seek states of higher value. Lower values (i.e., when $\delta = 0$ or 3) push the task more toward reinforcement learning, which in this dataset increases the difficulty and makes it hard for the RL agent to learn a strong policy. On the other hand, a higher value ($\delta = 7$) skews the problem toward supervised learning, causing the weight adjustments to depend almost entirely on prediction accuracy. This makes the task harder in practice and results in a weaker policy. It is observed in the experiments that the portfolio rebalancing performance is improved using the predicted MACD indicator. But the test dataset we chose for the portfolio across the COVID–19 pandemic. It might lead to two potential issues:

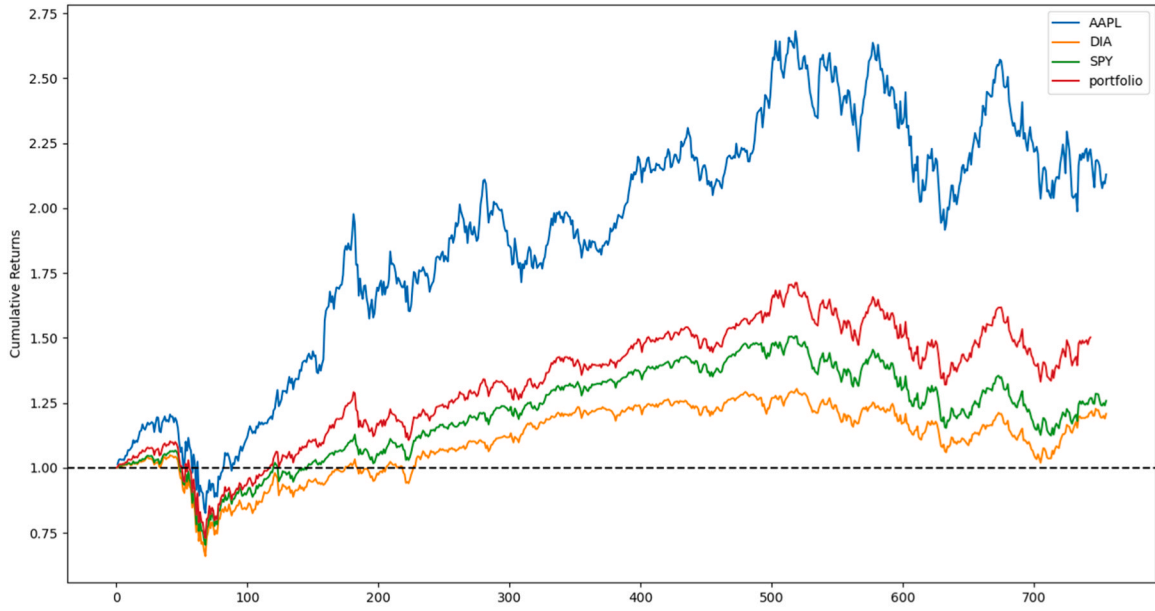
- COVID–19 is the most serious disaster, which crashed the global stock markets. It caused the occurrence of the concept drift, leading to significant differences between our training dataset and test dataset.
- During the period of COVID–19 pandemic, the technology stocks did not perform as badly as other sectors. AAPL was less impacted and experienced a significant increase in the stock prices.

As such, the RL agent made buy and sell decisions in situations that were different from what it had previously experienced. It results in the cumulative return of AAPL surpassing our method for most of the time points in the test dataset.

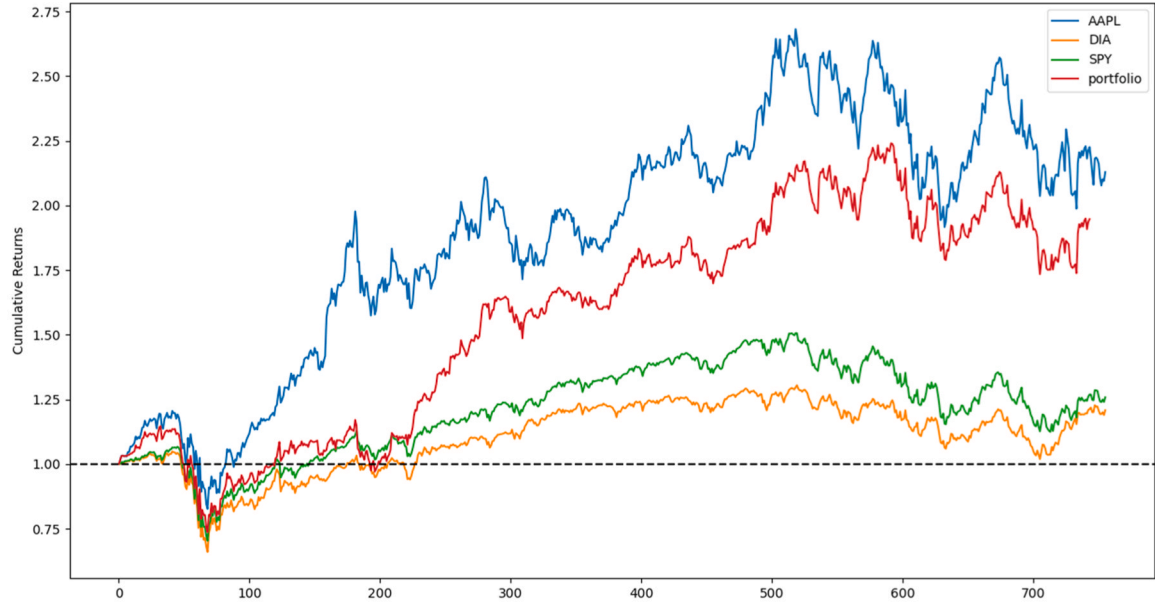
Table 10

Time period of the three financial assets in portfolio rebalancing experiment.

Data type	Time period
Train dataset	2002–08–19–2019–12–11
Test dataset	2019–12–12–2022–11–25



(a) NAV of individual assets and the portfolio by the RL agent with the vanilla Action



(b) NAV of individual assets and the portfolio by the RL agent with the revised Action

Fig. 7. Comparisons of the RL agent with the vanilla Action and revised Action.

4.5. Parametric study on time steps of fuzzy transformer and portfolio rebalancing

In this sub-section, two types of parametric study are performed. Firstly, the parametric study is conducted for the time step in the proposed fuzzy Transformer architecture, which caters for different input sequence lengths, i.e., the number of past daily observations fed into the Transformer. It is to investigate the impacts of time step values chosen in the fuzzy Transformer architecture. Secondly, the parametric study is conducted to investigate the relationship between trading time step intervals and stock holding period in the portfolio.

In the first parametric study, different values of the time step in the proposed fuzzy Transformer architecture are compared for predicting the n -day look-ahead prices of the financial asset AAPL. The study results are shown in Table 12. The time steps chosen for the fuzzy Transformer should be larger than 5 trading days. It is observed that time steps

between 10 and 15 achieve higher performance and relatively similar results within $t + 3$ look-ahead predictions. But continuing to increase the time step to be larger values does not necessarily lead to better performance. In the portfolio rebalancing tasks, the smaller time step value as 10 is chosen for inferring of the fuzzy Transformer, as it corresponds to 10 trading days, i.e., two-week calendar time frame.

We further conduct the parametric study on different trading time steps k for the RL agent, i.e., different portfolio rebalancing intervals. Observed in Fig. 9, when the trading time step k is set as 5, the model achieves the best performance in terms of cumulative return. To further investigate the relationship between the trading time step and the stock holding period in the portfolio, we log into the stock holding period during which significant portfolio weight changes occur (controlled by a threshold value; in our case, set to 0.1). However, naively logging the stock holding period cannot fully explain the relationship, as the holding period is at least equal to the portfolio rebalancing interval. Therefore,

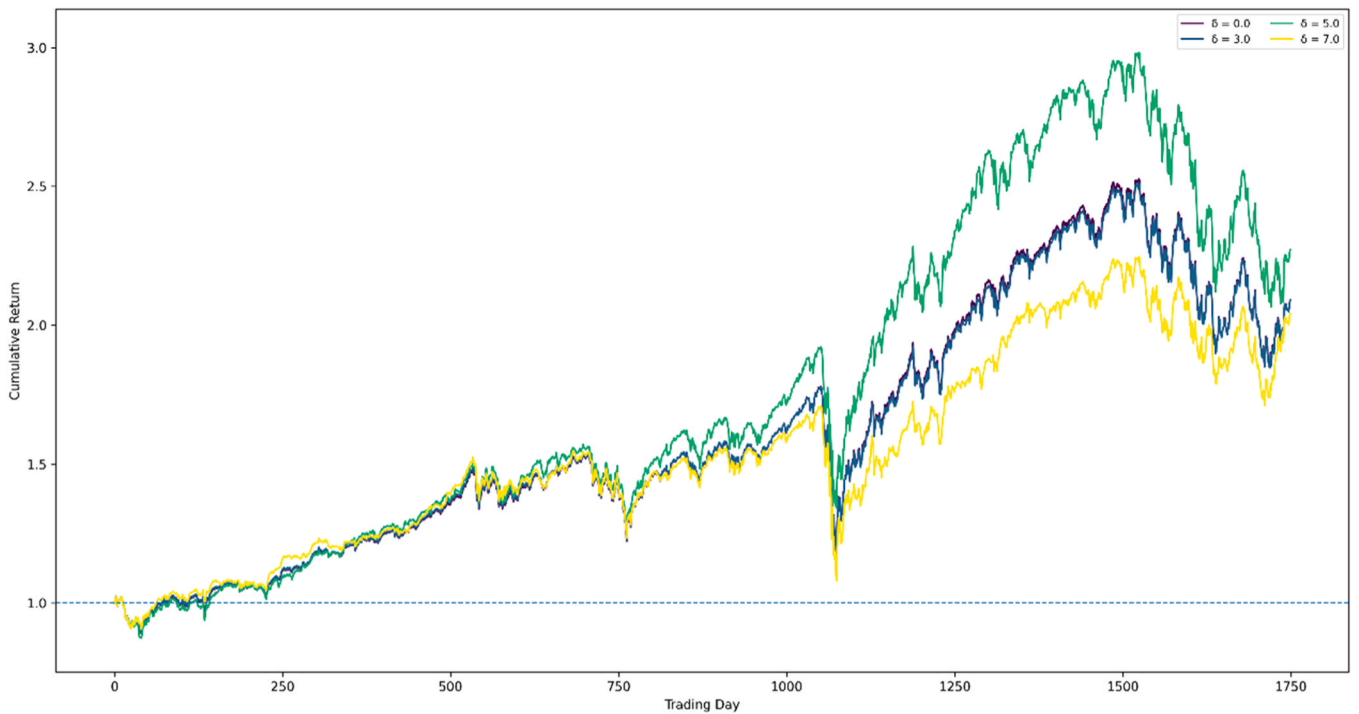


Fig. 8. Tuning the revised RL agent with different values of the hyper parameter δ .

Table 12

Parametric study with different time steps in the fuzzy Transformer under $t + 1$ to $t + 10$ look-ahead.

Look-ahead	Proposed fuzzy Transformer with different time steps (AAPL)							
	5 trading days		10 trading days		15 trading days		20 trading days	
	RMSE	R ²	RMSE	R ²	RMSE	R ²	RMSE	R ²
$t + 1$	1.6624	0.9953	0.8618	0.9969	0.7168	0.9971	0.6493	0.9978
$t + 2$	4.7379	0.9014	4.4431	0.9189	3.1516	0.9618	3.9411	0.9343
$t + 3$	8.8148	0.7613	4.0636	0.9322	4.8491	0.9146	4.5247	0.9219
$t + 4$	8.4165	0.7713	5.2173	0.8883	4.9892	0.8941	4.4846	0.9017
$t + 5$	10.1569	0.6716	6.5990	0.8212	6.7916	0.8457	6.2789	0.8753
$t + 6$	10.1895	0.6476	10.2684	0.5667	9.9481	0.6948	10.4961	0.5278
$t + 7$	10.4653	0.5135	10.6661	0.5321	11.7461	0.4894	12.1896	0.4943
$t + 8$	16.0651	0.1635	11.2682	0.4765	12.4986	0.4231	12.7325	0.4731
$t + 9$	20.6416	-0.6561	15.7418	-0.0242	14.9431	0.0156	15.1165	0.0018
$t + 10$	25.4167	-0.7447	18.2225	-0.3763	18.1983	-0.2794	20.4167	-0.4217

for larger portfolio rebalancing intervals, the stock holding period is naturally longer. To mitigate this correlation, we normalize the stock holding period by the trading time steps. The normalized correlation plot is shown in Fig. 10. It is observed that they are negatively correlated, with the average correlation value as about -0.70878 . When the RL agent operates under a longer portfolio rebalancing interval, it has already learned what constitutes correct or incorrect decisions within that setting. Therefore, once portfolio rebalancing is allowed, the RL agent tends to adjust and re-balance the portfolio weights through stock selling or buying transactions.

5. Conclusion and discussion

5.1. Conclusion

In this research, we first describe the rationale of the price predictions in order to derive a non-lagging MACD indicator. The predicted price by the proposed fuzzy Transformer architecture is utilized to compute the predicted MACD indicator. The trend reversal signals by the predicted MACD will be managed by a revised A2C RL agent, to rebalance the portfolio accordingly. The fuzzy Transformer architecture

combines the fuzzy logic part and the Transformer part. Both parts share the same input and output data values. The Transformer part is used to fit the input data to the labels. The fuzzy part determines the rules in the data with human understandable languages based on the input and output of Transformer. We can also identify the key features in the generated rules based on the attention layer. It helps the interpretability on deep learning models to better understand the underlying rules, and to some extent, the output results of deep learning models. We hope it can help increase the user confidence in adopting deep learning models.

The fuzzy Transformer architecture is able to predict the multi-day look-ahead stock prices in the financial market. Based on the predicted values, the predicted MACD can be calculated. It largely reduces the lagging problem of the vanilla MACD that is calculated using only historical values. Finally, we adopt an RL agent, but with some revisions made, including customizing the state vector, to cater for most relevant features such as predicted MACD values. We have also revised the Action function with tunable parameter and terms to aim at higher growth potential. We further revise the Reward function to include a tunable term of the penalty. The revised RL agent is trained by combining price information with the predicted MACD for a portfolio with three financial assets: DIA, SPY, and APPL.

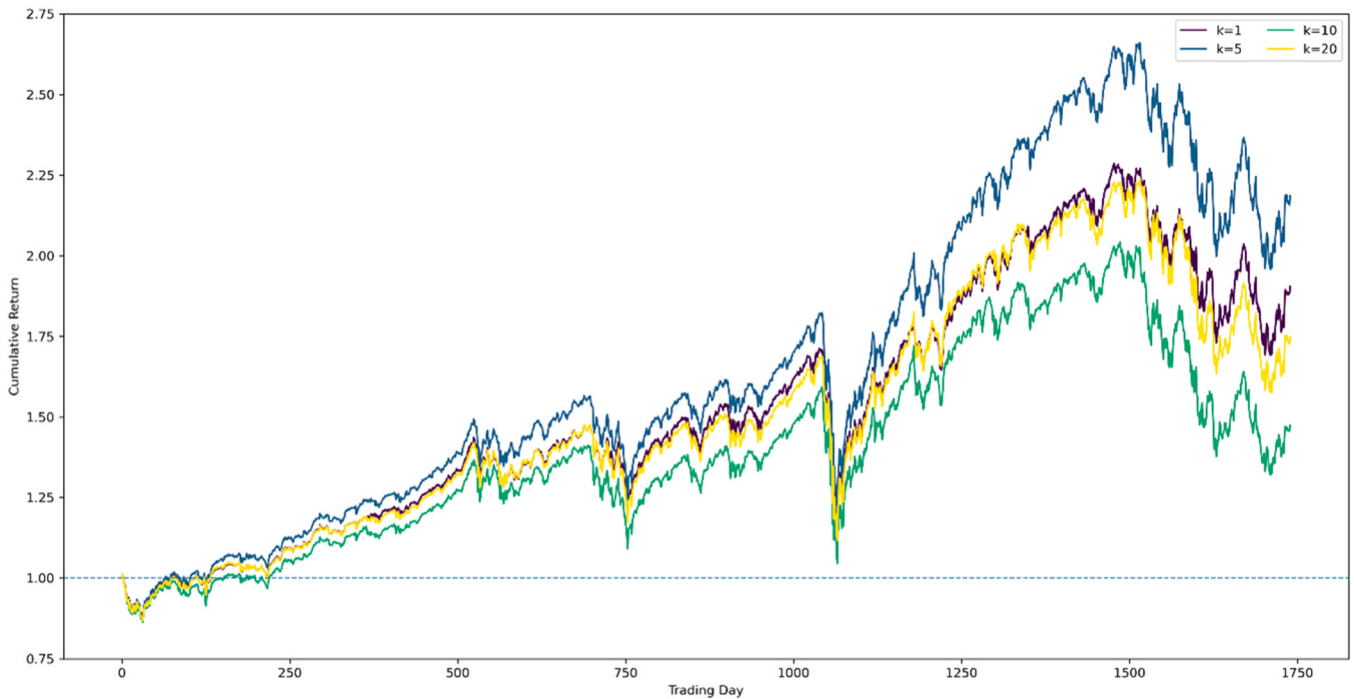


Fig. 9. Parametric study on different trading time steps for portfolio rebalancing.

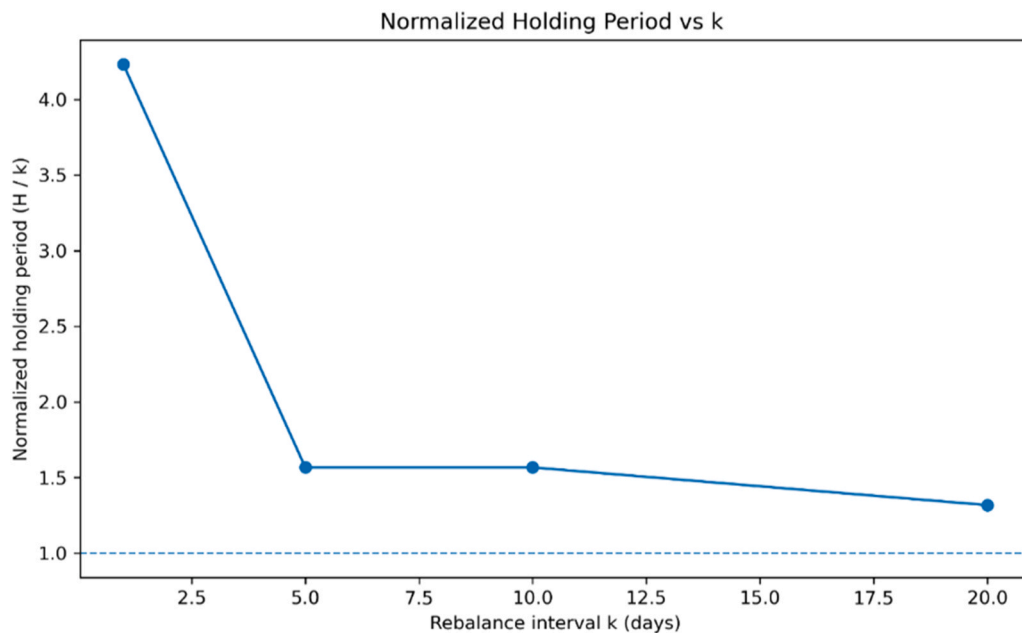


Fig. 10. Normalized correlation plot between stock holding period and portfolio rebalancing interval.

Extensive experiments have been conducted to evaluate the performance of predicting stock prices and rebalancing portfolio with the proposed fuzzy Transformer architecture and revised RL agent. We first discussed the feature selection by the revised MCES, based on the average feedback by four neural networks: MLP, CNN, LSTM and Transformer. It enables selecting most relevant features for each financial asset according to its historical data and patterns. Different features are selected and configured for different assets.

Next, the n -day look-ahead prices are predicted by the fuzzy Transformer architecture for each asset. The prediction performance is evaluated up to 10-day look-ahead. It is observed that the prediction performance decreases with the longer look-ahead days. The prediction

performance is also compared with several other deep learning algorithms, where the fuzzy Transformer achieves comparable performance.

With the predicted look-ahead prices, the predicted MACD indicator is derived to indicate the price trend change. The performance of the predicted MACD indicator by various deep learning networks are compared with that of the fuzzy Transformer. The fuzzy Transformer outperforms most of the benchmark networks.

The predicted MACD indicator is utilized by the A2C RL agent. In this work, we revise the Action and Rewards functions of RL agent, to make better decisions for higher rewards. The portfolio performance is also discussed with different parameter values. Different settings of the RL agent for portfolio rebalancing are experimented. Revising the Action

output of the RL agent makes some improvements in the cumulative return of the portfolio.

5.2. Discussions

The integration of attention weights into fuzzy rules opens a myriad of possibilities. But due to time constraints, this was not thoroughly explored in this work. In deep learning, it is challenging to define good features. One possible approach could be using attention weights to identify and eliminate irrelevant elements in antecedents, offering a potential alternative to the traditional feature selection methods. Another analytical approach is that some similar antecedents are found in the rules but leading to different consequence results. Based on the change of the attention scores, it can find out the state changes of which key features lead to different consequences.

Another limitation of this work is in trading and portfolio management. We rely only on the price data of the financial assets. But in real life, there are other factors to influence stock market movements, such as policy changes in a country, geopolitical shifts, societal issues, etc. The systems could benefit from leveraging other factors to predict stock market movements in the future.

CRedit authorship contribution statement

Qi Cao: Writing – review & editing, Validation, Methodology, Investigation. **Hui Jin:** Writing – original draft, Validation, Software, Formal analysis, Data curation. **Chai Quek:** Writing – review & editing, Supervision, Resources, Methodology, Conceptualization.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

Data will be made available on request.

References

- [1] A. Khan, M.M. Fouda, D. Do, et al., Short-term traffic prediction using deep learning long short-term memory: Taxonomy, applications, challenges, and future trends, *IEEE Access*. 11 (2023) 94371–94391, <https://doi.org/10.1109/ACCESS.2023.3309601>.
- [2] Z.Z. Lua, C.K. Seow, R.C.B. Chan, Y. Cai, Qi Cao, Automated Bitcoin Trading dApp using Price Prediction from a Deep Learning Model, *Risks* 13 (1) (2025), <https://doi.org/10.3390/risks13010017>.
- [3] H. Zhang, M.O. Shafiq, Survey of Transformers and towards ensemble learning using Transformers for natural language processing, *J. Big Data* 11 (2024), <https://doi.org/10.1186/s40537-023-00842-0>.
- [4] C. Liu, X. Chen, J. Lin, et al., Evolving masked low-rank Transformer for long text understanding, *Appl. Soft Comput.* 152 (2024), <https://doi.org/10.1016/j.asoc.2023.111207>.
- [5] A. Dash, M. Awachar, A. Patel, et al., Open-domain long-form question–answering using Transformer-based pipeline, *SN Comput. Sci.* 4 (2023), <https://doi.org/10.1007/s42979-023-02039-x>.
- [6] Z. Yu, F. Lee, Q. Chen, HCT-net: hybrid CNN-Transformer model based on a neural architecture search network for medical image segmentation, *Appl. Intell.* 53 (2023) 19990–20006, <https://doi.org/10.1007/s10489-023-04570-z>.
- [7] E. Arkin, N. Yadikar, X. Xu, et al., A survey: object detection methods from CNN to Transformer, *Multimed. Tools Appl.* 82 (2023) 21353–21383, <https://doi.org/10.1007/s11042-022-13801-3>.
- [8] C. Yang, X. Wang, L. Yao, Dyformer: A dynamic Transformer-based architecture for multivariate time series classification, *Inf. Sci.* 656 (2024), <https://doi.org/10.1016/j.ins.2023.119881>.
- [9] A.T. Haryono, R. Sarno, K.R. Sungkono, Transformer-gated recurrent unit method for predicting stock price based on news sentiments and technical indicators, *IEEE Access*. 11 (2023) 77132–77146, <https://doi.org/10.1109/ACCESS.2023.3298445>.
- [10] N.H.L. Kan, Q. Cao, C. Quek, Learning and Processing Framework using Fuzzy Deep Neural Network for Trading and Portfolio Rebalancing, *Appl. Soft Comput.* 152 (2024), <https://doi.org/10.1016/j.asoc.2024.111233>.
- [11] Q. Zhang, C. Qin, Y. Zhang, et al., Transformer-based attention network for stock movement prediction, *Expert. Syst. Appl.* 202 (2022), <https://doi.org/10.1016/j.eswa.2022.117239>.
- [12] C. Wang, Y. Chen, S. Zhang, Q. Zhang, Stock market index prediction using deep Transformer model, *Expert. Syst. Appl.* 208 (2022), <https://doi.org/10.1016/j.eswa.2022.118128>.
- [13] Z. Tao, W. Wu, J. Wang, Series decomposition Transformer with period-correlation for stock market index prediction, *Expert. Syst. Appl.* 237 (2024), <https://doi.org/10.1016/j.eswa.2023.121424>.
- [14] P. Qiu, Z. Niu, TCIC-FS: Total correlation information coefficient-based feature selection method for high-dimensional data, *Knowl. -Based Syst.* 231 (2021), <https://doi.org/10.1016/j.knsys.2021.107418>.
- [15] G.A. Nemnes, N. Filipoiu, V. Sipica, Feature selection procedures for combined density functional theory—artificial neural network schemes, *Phys. Scr.* 96 (6) (2021), <https://doi.org/10.1088/1402-4896/abf3f7>.
- [16] Z. Noroozi, A. Orooji, L. Erfannia, Analyzing the impact of feature selection methods on machine learning algorithms for heart disease prediction, *Sci. Rep.* 13 (2023), <https://doi.org/10.1038/s41598-023-49962-w>.
- [17] K.H. Quah, C. Quek, MCES: A Novel Monte Carlo Evaluative Selection Approach for Objective Feature Selections, *IEEE Trans. Neural Netw.* 18 (2) (2007) 431–448, <https://doi.org/10.1109/TNN.2006.887555>.
- [18] T. Tu, Y. Su, Y. Tang, W. Tan, S. Ren, A more flexible and robust feature selection algorithm, *IEEE Access*. 11 (2023) 141512–141522, <https://doi.org/10.1109/ACCESS.2023.3342044>.
- [19] L. Meneghetti, N. Demo, G.A. Rozza, Dimensionality reduction approach for convolutional neural networks, *Appl. Intell.* 53 (2023) 22818–22833, <https://doi.org/10.1007/s10489-023-04730-1>.
- [20] V. Hassija, V. Chamola, A. Mahapatra, et al., Interpreting black-box models: A review on explainable artificial intelligence, *Cogn. Comput.* 16 (2024) 45–74, <https://doi.org/10.1007/s12559-023-10179-8>.
- [21] J.C.M. Tan, Q. Cao, C. Quek, FE-RNN: A Fuzzy Embedded Recurrent Neural Network for improving interpretability of underlying neural network, *Inf. Sci.* 663 (2024), <https://doi.org/10.1016/j.ins.2024.120276>.
- [22] X. Gu, J. Han, Q. Shen, et al., Autonomous learning for fuzzy systems: a review, *Artif. Intell. Rev.* 56 (2023) 7549–7595, <https://doi.org/10.1007/s10462-022-10355-6>.
- [23] A. Salimi-Badr, A data-driven implicit deep adaptive neuro-fuzzy inference system capable of manifold learning for function approximation, *Appl. Soft Comput.* 155 (2024), <https://doi.org/10.1016/j.asoc.2024.111458>.
- [24] X. Meng, Y. Zhang, L. Quan, J. Qiao, A self-organizing fuzzy neural network with hybrid learning algorithm for nonlinear system modeling, *Inf. Sci.* 642 (2023), <https://doi.org/10.1016/j.ins.2023.119145>.
- [25] L. Su, X. Zuo, R. Li, et al., A systematic review for Transformer-based long-term series forecasting, *Artif. Intell. Rev.* 58 (80) (2025), <https://doi.org/10.1007/s10462-024-11044-2>.
- [26] R. Venugopal, C. Veeramani, S. Muruganandan, An effective approach for predicting daily stock trading decisions using fuzzy inference systems, *Soft Comput.* 28 (2024) 3301–3319, <https://doi.org/10.1007/s00500-023-09383-3>.
- [27] X. Wu, H. Chen, J. Wang, et al., Adaptive stock trading strategies with deep reinforcement learning methods, *Inf. Sci.* 538 (2020) 142–158, <https://doi.org/10.1016/j.ins.2020.05.066>.
- [28] M. Melda, M. Mira, N. Nurlina, Analysis of the Effectiveness of RSI and MACD Indicators in Addressing Stock Price Volatility, *Wiga J. Penelit. Ilmu Ekon.* 15 (1) (2025) 71–79, <https://doi.org/10.30741/wiga.v15i1.1449>.
- [29] A. Guarino, D. Santoro, L. Grilli, L. et al., EvoFolio: a portfolio optimization method based on multi-objective evolutionary algorithms, *Neural Comput. Appl.* 36 (2024) 7221–7243, <https://doi.org/10.1007/s00521-024-09456-w>.
- [30] R. Abdalla, W. Hollstein, C.P. Carvajal, et al., Actor-critic reinforcement learning leads decision-making in energy systems optimization—steam injection optimization, *Neural Comput. Appl.* 35 (2023) 16633–16647, <https://doi.org/10.1007/s00521-023-08537-6>.
- [31] J. Park, S. Han, Reinforcement learning with multimodal advantage function for accurate advantage estimation in robot learning, *Eng. Appl. Artif. Intell.* 126 (2023), <https://doi.org/10.1016/j.engappai.2023.107019>.
- [32] S. Chakraborty, F. Heintz, Enhancing Time Series Forecasting with Fuzzy Attention-Integrated Transformers, *arXiv* (2025), <https://doi.org/10.48550/arXiv.2504.00070>.
- [33] E. E. A.R. S. A. Balakrishnan, et al., Stacked hybrid model for load forecasting: integrating Transformers, ANN, and fuzzy logic, *Sci. Rep.* 15 (2025) 19688, <https://doi.org/10.1038/s41598-025-04210-1>.
- [34] E.A. Engel, N.E. Engel, A Transformer with a Fuzzy Attention Mechanism for Weather Time Series Forecasting, *Advances in Neural Computation, Machine Learning, and Cognitive Research VIII*, Springer, Cham, 2024, https://doi.org/10.1007/978-3-031-80463-2_39.
- [35] P. Dubey, P. Dubey, Bridging spatiotemporal wildfire prediction and decision modeling using Transformer networks and fuzzy inference systems, *MethodsX* 15 (2025), <https://doi.org/10.1016/j.mex.2025.103498>.
- [36] L. Zhao, Y. Li, X. Chen, et al., MFTM-Informer: A multi-step prediction model based on multivariate fuzzy trend matching and Informer, *Inf. Sci.* 681 (2024), <https://doi.org/10.1016/j.ins.2024.121268>.
- [37] X. Chen, L. Lai, M. Luo, FDACNet: Enhancing time-series classification with fuzzy feature and integrated self-attention and temporal convolution, *Int. J. Approx. Reason.* 186 (2025), <https://doi.org/10.1016/j.ijar.2025.109521>.
- [38] İ. Özdoğan, O. Yıldız, F.E. Boran, A hybrid fuzzy-CNN model with feature extraction from MACD-based trends for stock market movement forecasting, *Expert. Syst. Appl.* 296 (2026), <https://doi.org/10.1016/j.eswa.2025.129068>.

- [39] J. Timko, R.E. Shawi, S. Tomasiello, Optimizing stock price forecasting: a hybrid approach using fuzziness and automated machine learning, *Expert. Syst. Appl.* 295 (2026), <https://doi.org/10.1016/j.eswa.2025.128844>.
- [40] E.M. Amiri, A. Esfahanipour, An optimized convolutional neural network with a novel spherical triangular fuzzy pooling layer for an algorithmic trading model, *Appl. Soft Comput.* 182 (2025), <https://doi.org/10.1016/j.asoc.2025.113617>.
- [41] A. Hadizadeh, M.J. Tarokh, M.M. Ghazani, A novel Transformer-based dual attention architecture for the prediction of financial time series, *J. King Saud. Univ. Comput. Inf. Sci.* 37 (72) (2025), <https://doi.org/10.1007/s44443-025-00045-y>.
- [42] K. Yemets, I. Izonin, I. Dronyuk, Time Series Forecasting Model Based on the Adapted Transformer Neural Network and FFT-Based Features Extraction, *Sensors* 25 (3) (2025) 652, <https://doi.org/10.3390/s25030652>.
- [43] Y. Xue, S. Guan, W. Jia, PMformer: A novel informer-based model for accurate long-term time series prediction, *Inf. Sci.* 690 (2025), <https://doi.org/10.1016/j.ins.2024.121586>.
- [44] S. Huang, Y. Liu, H. Cui, et al., MEAformer: An all-MLP Transformer with temporal external attention for long-term time series forecasting, *Inf. Sci.* 669 (2024), <https://doi.org/10.1016/j.ins.2024.120605>.
- [45] T. Bashir, H. Wang, M. Tahir, Y. Zhang, Wind and solar power forecasting based on hybrid CNN-ABiLSTM, CNN-Transformer-MLP models, *Renew. Energy* 239 (2025), <https://doi.org/10.1016/j.renene.2024.122055>.
- [46] J. Xiao, B. Long, A multi-channel spatial-temporal Transformer model for traffic flow forecasting, *Inf. Sci.* 671 (2024), <https://doi.org/10.1016/j.ins.2024.120648>.
- [47] A.K. Mishra, J. Renganathan, A. Gupta, Volatility forecasting and assessing risk of financial markets using multi-Transformer neural network based architecture, *Eng. Appl. Artif. Intell.* 133 (2024), <https://doi.org/10.1016/j.engappai.2024.108223>.
- [48] X. Qi, Z. Xu, F. Wang, Temporal fusion point-interval forecasting: A comprehensive approach for financial time series prediction, *Appl. Soft Comput.* 169 (2025), <https://doi.org/10.1016/j.asoc.2024.112600>.
- [49] A.M. Ikotun, A.E. Ezugwu, L. Abualigah, et al., K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data, *Inf. Sci.* 622 (2023) 178–210, <https://doi.org/10.1016/j.ins.2022.11.139>.
- [50] N. Monath, M. Zaheer, A. McCallum, Online level-wise hierarchical clustering, 29th ACM SIGKDD Conf. Knowl. Discov. Data Min. (2023) 1733–1745, <https://doi.org/10.1145/3580305.3599455>.
- [51] X. Xie, L. Duan, T. Qiu, et al., Quantum algorithm for MMNG-based DBSCAN, *Sci. Rep.* 11 (2021) 15559, <https://doi.org/10.1038/s41598-021-95156-7>.
- [52] S.R. Kasa, V. Rajan, Avoiding inferior clusterings with misspecified Gaussian mixture models, *Sci. Rep.* 13 (2023) 19164, <https://doi.org/10.1038/s41598-023-44608-3>.
- [53] Y. Qian, S. Yao, T. Wu, et al., Improved selective deep-learning-based clustering ensemble, *Appl. Sci.* 14 (2) (2024), <https://doi.org/10.3390/app14020719>.
- [54] K. Berahmand, Y. Li, Y. Xu, DAC-HPP: deep attributed clustering with high-order proximity preserve, *Neural Comput. Appl.* 35 (2023) 24493–24511, <https://doi.org/10.1007/s00521-023-09052-4>.
- [55] X. Wei, Z. Zhang, H. Huang, Y. Zhou, An overview on deep clustering, *Neurocomputing* (2024), <https://doi.org/10.1016/j.neucom.2024.127761>.
- [56] L.L.X. Yeo, Q. Cao, C. Quek, Dynamic portfolio rebalancing with lag-optimised trading indicators using SeroFAM and genetic algorithms, *Expert. Syst. Appl.* 216 (2023), <https://doi.org/10.1016/j.eswa.2022.119440>.
- [57] G. Zhong, F. Liu, J. Jiang, B. Wang, C.L.P. Chen, Refining one-class representation: A unified Transformer for unsupervised time-series anomaly detection, *Inf. Sci.* 656 (2024), <https://doi.org/10.1016/j.ins.2023.119914>.
- [58] Y. Li, N. Miao, L. Ma, et al., Transformer for object detection: Review and benchmark, *Eng. Appl. Artif. Intell.* 126 (2023), <https://doi.org/10.1016/j.engappai.2023.107021>.
- [59] AI4Finance Foundation, FinRL Library, [Online]. Available: (<https://finrl.readthedocs.io/en/latest/index.html>). [Accessed 5 Feb. 2023].
- [60] C. Deng, Y. Huang, N. Hasan, Y. Bao, Multi-step-ahead stock price index forecasting using long short-term memory model with multivariate empirical mode decomposition, *Inf. Sci.* 607 (2022) 297–321.